LAPORAN MILESTONE 3 TUGAS BESAR

IF2224 TEORI BAHASA FORMAL DAN AUTOMATA

Semantic Analysis

Disusun oleh:

JBC - J1B2C2

Samuelson D. Tanuraharja	13524001
Aufa Rienaldifaza Ahmad	13524027
Niko Samuel Simanjuntak	13524029
Aurelia Jennifer Gunawan	13524089
Reinsen Silitonga	13524093

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESA 10, BANDUNG 40132

2026

DAFTAR ISI

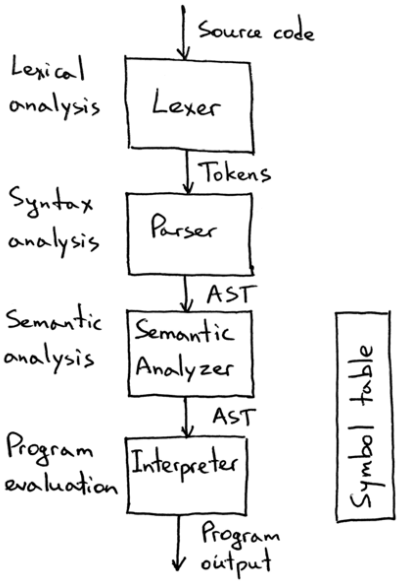
DAFTAR GAMBAR

DAFTAR TABEL

BAB 1
TEORI SINGKAT

1.1. Semantic Analysis

Semantic analysis adalah tahap ketiga pada sistem compiler yang bertujuan untuk validasi makna program. Pada tahap analisis leksikal dan analisis sintaks, pernyataan dan deklarasi dalam program hanya divalidasi secara struktur bahasanya, meliputi kebenaran leksikal dan grammar. Program mungkin masih mengandung pernyataan-pernyataan yang tidak logis ketika dieksekusi. Pada tahap analisis semantik, *compiler* tidak hanya memastikan bahwa struktur program sudah sesuai grammar, tetapi juga memastikan bahwa program memiliki arti yang valid secara logis dan tipe data. *Semantic analysis* dilakukan dengan menelusuri *parse tree* atau *abstract syntax tree* (AST) untuk memeriksa aturan semantik bahasa pemrograman.



Gambar 1. Alur Sistem dari Lexer Hingga Interpreter

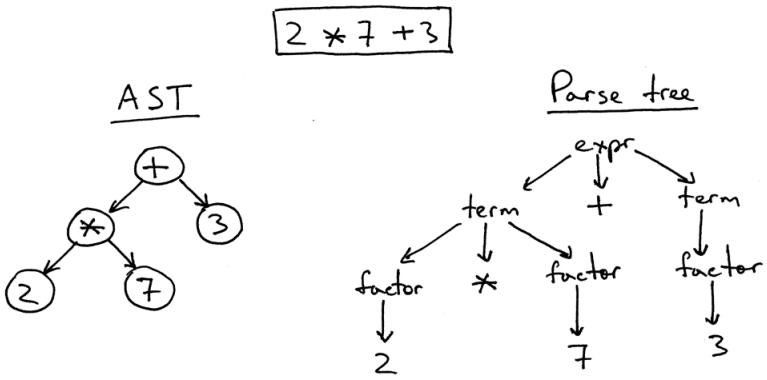
Beberapa proses utama dalam *semantic analysis* meliputi *type checking*, *symbol table management*, *scope resolution*, dan *control flow validation*. *Type checking* digunakan untuk memastikan operator dan operan memiliki tipe data yang kompatibel, misalnya *integer* tidak dapat dijumlahkan dengan *string*. *Symbol table management* digunakan untuk memastikan setiap *identifier* telah dideklarasikan sebelum digunakan. *Scope resolution* berperan dalam memvalidasi

akses variabel berdasarkan cakupan program. Sementara itu, *control flow validation* memastikan struktur kontrol program berjalan dengan benar.

1.2. Abstract Syntax Tree

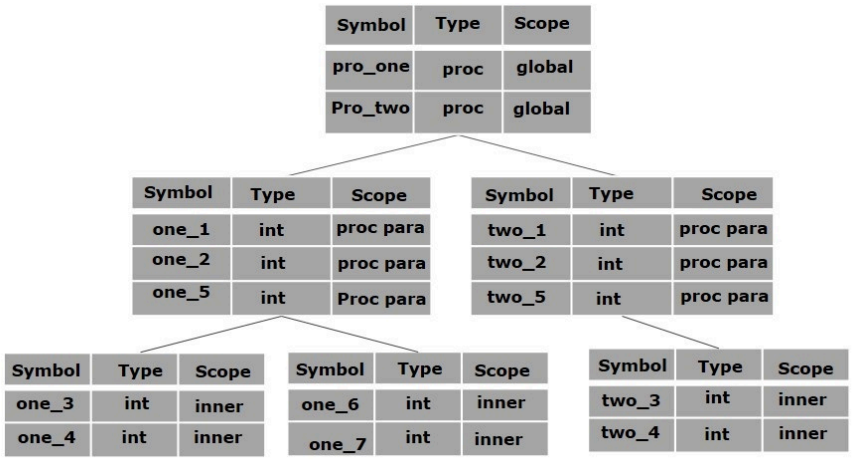
Abstract Syntax Tree (AST) adalah representasi struktur sintaks program dalam bentuk pohon yang lebih sederhana dibandingkan *parse tree*. AST hanya menyimpan informasi penting yang dibutuhkan pada tahap selanjutnya dalam sistem *compiler*. Informasi sintaksis yang tidak diperlukan, seperti tanda kurung, *keyword* tertentu, atau simbol pemisah, dihilangkan sehingga AST lebih berfokus pada hubungan semantik antar operator dan operan dibandingkan aturan *grammar* secara lengkap.

AST dibentuk dengan cara menelusuri parse tree hasil analisis sintaks lalu menyederhanakannya menjadi node yang merepresentasikan operasi utama dalam program, seperti operasi *assignment*, operasi aritmatika, percabangan, dan perulangan. Berbeda dengan parse tree yang cenderung lebih detail dan mengandung banyak informasi *grammar* yang tidak lagi diperlukan setelah tahap *parsing* selesai, AST dapat membuat analisis semantik menjadi lebih efisien. Selain itu, AST juga menjadi dasar untuk pembentukan *decorated abstract syntax tree*, yaitu AST yang telah diberi anotasi tambahan seperti tipe data, *lexical scope*, dan referensi *symbol table*.



Gambar 2. Contoh Perbandingan AST dan Parse Tree

1.3. Symbol Table



Gambar 3. Contoh Symbol Table

Symbol table merupakan struktur data yang digunakan compiler untuk menyimpan informasi atribut dari setiap *identifier*, seperti nama variabel, tipe data, *scope*, alamat memori, dan jenis objek. *Symbol table* berperan sebagai pusat penyimpanan informasi yang digunakan sistem compiler dalam proses analisis semantik. Beberapa fungsi utama dari *symbol table* adalah sebagai berikut:

1. Validasi keberadaan identifier

   Compiler harus memastikan bahwa setiap *identifier* telah dideklarasikan sebelum digunakan sebagai variabel, konstanta, maupun fungsi. *Symbol table* digunakan sebagai *lookup table* untuk memeriksa status deklarasi *identifier*. Jika *identifier* tidak ditemukan dalam tabel, compiler akan menghasilkan *undeclared identifier error*. Namun, jika *identifier* ditemukan lebih dari satu kali pada *scope* yang sama, compiler akan menghasilkan *multiple declaration error*.

2. Type checking

   Ketika *semantic analyzer* menemukan ekspresi operasi, seperti operasi aritmatika, *assignment*, dan logika, compiler akan memeriksa kompatibilitas tipe data operan yang terlibat. Proses ini bertujuan mencegah kesalahan semantik, seperti penggunaan operasi aritmatika pada tipe data yang tidak kompatibel seperti *string* atau karakter. Informasi tipe data yang tersimpan dalam *symbol table* digunakan sebagai dasar validasi tersebut.

3. Scope checking

   Bahasa Arion menggunakan konsep *block structure*, sehingga setiap *identifier* memiliki cakupan akses atau *scope* tertentu. *Identifier* yang dideklarasikan pada suatu *block* hanya dapat diakses di dalam *block* tersebut dan *block* turunannya, sedangkan *identifier* global dapat diakses dari seluruh bagian program. Ketika *semantic analyzer* melakukan penelusuran *Abstract Syntax Tree* (AST), compiler akan membuka *scope* baru setiap kali memasuki *block* seperti prosedur, fungsi, atau *statement* majemuk. Proses pencarian *identifier* dilakukan mulai dari *scope* terdalam menuju *scope* terluar. Dengan mekanisme ini, compiler dapat memastikan bahwa setiap *identifier* diakses pada cakupan yang valid dan mencegah konflik antar *identifier* dengan nama yang sama pada *block* berbeda.

4. Penyimpanan metadata

   Dalam implementasinya, *symbol table* dapat terdiri atas beberapa jenis tabel, seperti tab, atab (*array table*), dan btab (*block table*). Tabel tab digunakan untuk menyimpan informasi *identifier* seperti nama, tipe data, referensi, dan *scope*. Tabel atab menyimpan *metadata array* seperti tipe indeks, batas bawah dan atas indeks, tipe elemen, serta ukuran *array*. Sementara itu, btab digunakan untuk menyimpan *metadata block* seperti prosedur atau fungsi, termasuk ukuran *parameter* dan variabel lokal pada *block* tersebut.

1.4. L-Attributed Grammar

L-Attributed Grammar merupakan salah satu bentuk *attributed grammar* yang digunakan dalam proses *semantic analysis* pada compiler. *Attributed grammar* adalah *grammar* yang dilengkapi dengan atribut dan aturan semantik untuk membantu compiler melakukan evaluasi makna program. Setiap simbol pada *grammar* dapat memiliki atribut tertentu yang menyimpan informasi semantik.

L-Attributed Grammar termasuk ke dalam skema *Syntax-Directed Translation* (SDT) yang mengatur bahwa atribut pada suatu node dapat berupa *inherited attribute* ataupun *synthesized attribute*. *Inherited attribute* adalah atribut yang nilainya diturunkan dari *parent node* atau diperoleh dari *sibling node* di sebelah kiri dalam suatu aturan produksi. Sementara itu, *synthesized attribute* merupakan atribut yang nilainya diperoleh dari hasil evaluasi *child node* dan dikembalikan ke *parent node*.

Secara formal, pada aturan produksi $A \rightarrow X_1 X_2 \ldots X_n$, atribut *inherited* dari simbol X_j hanya boleh bergantung pada:

1. atribut milik non-terminal A sebagai *parent node*, dan
2. Atribut simbol-simbol $X_1, X_2, \ldots, X_{j-1}$ yang berada di sebelah kirinya.

Aturan ini memungkinkan evaluasi atribut dilakukan secara berurutan dari kiri ke kanan selama proses *parsing* berlangsung.

Dalam implementasi sistem compiler, *L-Attributed Grammar* digunakan untuk membantu *semantic analyzer* melakukan propagasi informasi semantik saat traversal *Abstract Syntax Tree* (AST). Informasi *scope*, *lexical level*, dan tipe data dapat diwariskan menggunakan *inherited attribute*, sedangkan hasil evaluasi ekspresi atau *statement* dapat disimpan menggunakan *synthesized attribute*. Pendekatan ini mempermudah proses *semantic checking* tanpa memerlukan traversal tambahan terhadap struktur pohon program.

BAB 2
PERANCANGAN DAN IMPLEMENTASI

2.1.Teknologi yang Digunakan

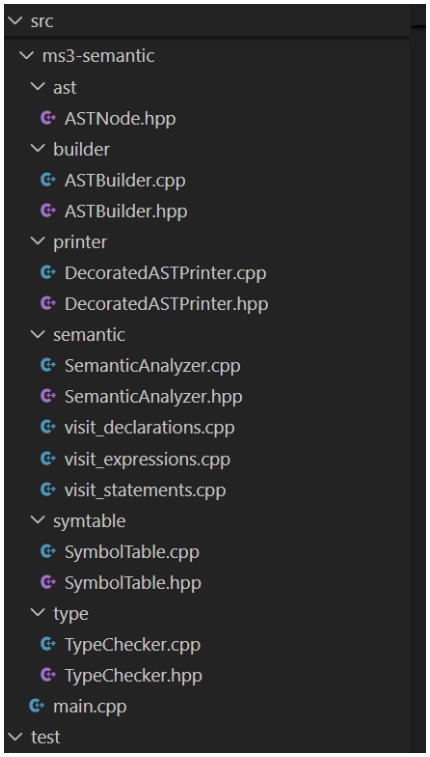
Program *semantic analyzer* ditulis dengan bahasa pemrograman C++ (standar C++11 atau lebih tinggi) tanpa dependensi dari pihak ketiga. Pemilihan C++ didasarkan pada kebutuhan akan performa tinggi dalam pemrosesan token dan fleksibilitas dalam pengelolaan memori secara eksplisit. Lingkungan pengembangan yang digunakan bersifat portabel, dengan dukungan *build tool* CMake versi minimal 3.16 untuk menjamin kompilasi lintas *platform*. Program bekerja dalam tiga tahap, yaitu tokenisasi, *parsing*, konstruksi AST, dan analisis semantik. Tokenisasi dilakukan oleh *lexical analyzer* yang telah dikembangkan pada milestone 1. Sedangkan *parsing* dengan pendekatan *recursive descent* yang telah dikembangkan pada milestone 2 menghasilkan *parse tree* sebagai representasi awal struktur program. Lalu, *parse tree* dikonversi menjadi *abstract syntax tree* (AST) oleh modul ASTBuilder dengan struktur *node* untuk deklarasi, *statement*, ekspresi, dan tipe data. Analisis semantik dilakukan oleh *semantic analyzer* menggunakan pola *Visitor* yang dipisahkan ke dalam modul deklarasi, *statement*, dan ekspresi. Informasi semantik dikelola melalui tabel simbol tab, atab, dan btab, sedangkan pemeriksaan tipe ditangani oleh TypeChecker. Setiap *node* AST menyimpan SemanticInfo hasil anotasi analisis. Output akhir berupa *decorated* AST dan *dump symbol table*.

2.2.Struktur Program

Arsitektur program *semantic analyser* dirancang secara modular dengan pemisahan tanggung jawab yang jelas antar komponen. Komponen utama terdiri dari ASTnode, ASTBuilder, SemanticAnalyzer, SymbolTable, TypeChecker, dan DecoratedASTPrinter. Struct ASTNode beserta seluruh turunannya merepresentasikan simpul pada *abstract syntax tree* (AST) dengan atribut *kind*, nomor baris, dan *field* SemanticInfo yang menyimpan hasil anotasi semantik berupa kode tipe, indeks tabel simbol, level lexical, dan referensi ke tabel pendukung. Kelas ASTBuilder bertugas mengonversi *parse tree* yang dihasilkan oleh *parser* menjadi AST yang lebih ringkas dengan memetakan setiap aturan *grammar* ke node AST yang sesuai. Kelas SemanticAnalyzer menjadi komponen sentral yang mengorkestrasikan seluruh proses analisis dengan pola Visitor, di mana implementasinya dibagi ke dalam tiga berkas terpisah sesuai kategori node, yaitu visit_declarations.cpp, visit_statements.cpp, dan visit_expressions.cpp. Kelas SymbolTable mengelola tiga tabel internal, yakni tab untuk entri simbol, atab untuk metadata tipe array, dan

btab untuk blok dan cakupan leksikal. Kelas TypeChecker menangani seluruh logika *type checking* secara terpisah dari traversal AST. Kelas DecoratedASTPrinter mencetak hasil AST yang telah dianotasi bersama dump ketiga *symbol table* sebagai output.

Alur data dimulai dari *lexer* yang menghasilkan token, kemudian diproses oleh *parser* untuk menghasilkan *parse tree* yang selanjutnya dikonversi oleh ASTBuilder menjadi AST melalui traversal *depth-first*. Lalu, AST dianalisis oleh SemanticAnalyzer yang memulai traversal dari visitProgram secara rekursif, berinteraksi dengan SymbolTable untuk mendaftarkan atau mencari simbol, dan memanggil TypeChecker untuk memvalidasi kesesuaian tipe di setiap *node*. Pengelolaan cakupan leksikal dilakukan melalui mekanisme openBlock dan closeBlock berbasis *stack*, sementara *error* dikumpulkan ke dalam sebuah *vector of string* tanpa menghentikan traversal sehingga seluruh kesalahan dapat dilaporkan di akhir eksekusi.



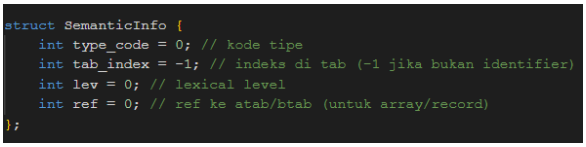
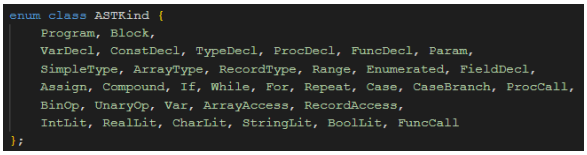
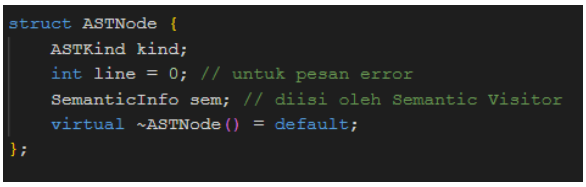
Gambar 2.3.2 Struktur Direktori Program *Semantic Analyser*

2.3.Fungsi/Kelas Utama dan Implementasinya

2.4.1. File ASTNode.hpp dan ASTNode.cpp

Tabel 2.4.1. Implementasi kelas ASTNode

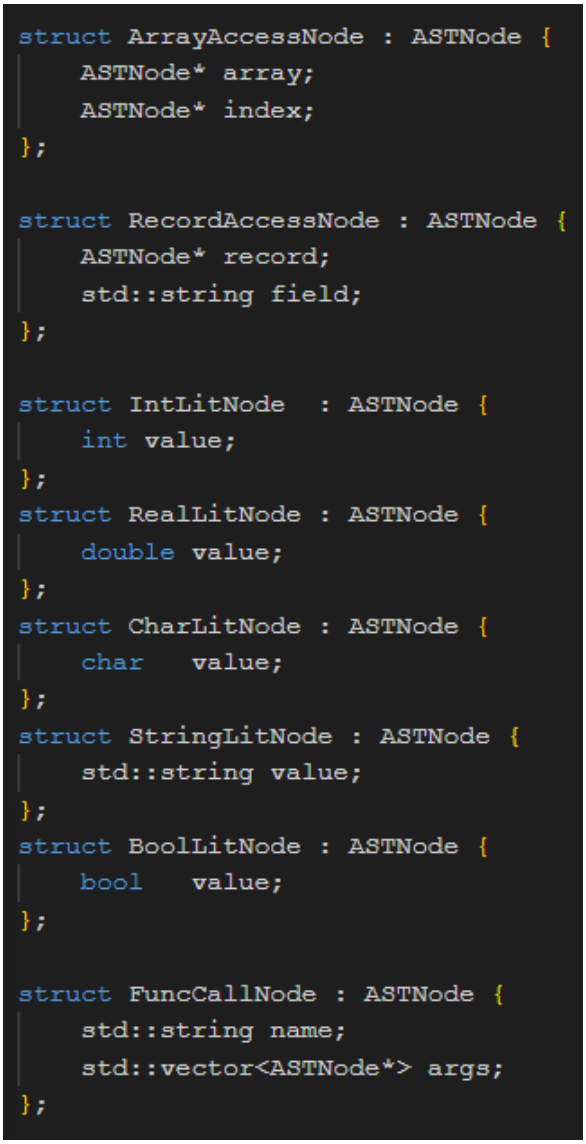
Kelas/Fungsi/Prosedur	**Penjelasan**

struct SemanticInfo  	Struktur data untuk menyimpan anotasi informasi semantik pada setiap ekspresi di dalam node AST. Informasi yang disimpan mencakup kode tipe objek, indeks pada tabel simbol, tingkat leksikal (lexical level), dan referensi ke tabel array atau record.
enum class ASTKind  	Enum dasar yang mendefinisikan seluruh jenis node yang terdapat di dalam program, mulai dari hierarki blok program, bagian deklarasi, tipe data, statement, hingga ekspresi spesifik.
struct ASTNode  	Kelas basis (base class) polimorfik untuk semua node di dalam tree. Struktur ini memuat variabel kind untuk menandai tipe node, parameter line untuk melakukan tracking lokasi pembentukan pesan error, dan objek sem sebagai wadah anotasi oleh Semantic Visitor.
struct ProgramNode, BlockNode, VarDeclNode, ConstDeclNode, TypeDeclNode, ParamNode, ProcDeclNode, FuncDeclNode, SimpleTypeNode, RangeNode, ArrayTypeNode, FieldDeclNode, RecordTypeNode, EnumeratedNode, CompoundNode, AssignNode, IfNode, WhileNode, ForNode, RepeatNode, CaseBranchNode, CaseNode, ProcCallNode, BinOpNode, UnaryOpNode, VarNode, ArrayAccessNode, RecordAccessNode, IntLitNode, RealLitNode, CharLitNode, StringLitNode, BoolLitNode, FuncCallNode	Kumpulan struktur kelas turunan konkret dari ASTNode yang merepresentasikan komponen bahasa Arion. Setiap struct didefinisikan dengan atribut pointer tambahan ke anakannya sesuai dengan aturan produksinya masing-masing, sehingga dapat membangun hierarki AST yang lengkap untuk proses traversal.

```cpp
struct ProgramNode : ASTNode {
    std::string name;
    std::vector<ASTNode*> declarations;
    ASTNode* body;
};

struct BlockNode : ASTNode {
    std::vector<ASTNode*> declarations;
    ASTNode* body;
};

struct VarDeclNode : ASTNode {
    std::vector<std::string> names;
    ASTNode* typeNode;
};

struct ConstDeclNode : ASTNode {
    std::string name;
    ASTNode* value;
};

struct TypeDeclNode : ASTNode {
    std::string name;
    ASTNode* typeNode;
};

struct ParamNode : ASTNode {
    std::vector<std::string> names;
    ASTNode* typeNode;
    bool isByRef = false;
};

struct ProcDeclNode : ASTNode {
    std::string name;
    std::vector<ParamNode*> params;
    BlockNode* block;
};

struct FuncDeclNode : ASTNode {
    std::string name;
    std::vector<ParamNode*> params;
    std::string returnTypeName;
    BlockNode* block;
};
```

```cpp
struct SimpleTypeNode : ASTNode {
    std::string typeName;
};

struct RangeNode : ASTNode {
    ASTNode* low;
    ASTNode* high;
};

struct ArrayTypeNode : ASTNode {
    ASTNode* indexType;
    ASTNode* elementType;
};

struct FieldDeclNode : ASTNode {
    std::vector<std::string> names;
    ASTNode* typeNode;
};

struct RecordTypeNode : ASTNode {
    std::vector<FieldDeclNode*> fields;
};

struct EnumeratedNode : ASTNode {
    std::vector<std::string> values;
};

struct CompoundNode : ASTNode {
    std::vector<ASTNode*> statements;
};

struct AssignNode : ASTNode {
    ASTNode* target;
    ASTNode* value;
};

struct IfNode : ASTNode {
    ASTNode* condition;
    ASTNode* thenBranch;
    ASTNode* elseBranch = nullptr;
};

struct WhileNode : ASTNode {
    ASTNode* condition;
    ASTNode* body;
};
```

```cpp
struct ForNode : ASTNode {
    std::string varName;
    ASTNode* fromExpr;
    ASTNode* toExpr;
    bool downto = false;
    ASTNode* body;
};

struct RepeatNode : ASTNode {
    std::vector<ASTNode*> body;
    ASTNode* condition;
};

struct CaseBranchNode : ASTNode {
    std::vector<ASTNode*> labels;
    ASTNode* statement;
};

struct CaseNode : ASTNode {
    ASTNode* expr;
    std::vector<CaseBranchNode*> branches;
};

struct ProcCallNode : ASTNode {
    std::string name;
    std::vector<ASTNode*> args;
};

struct BinOpNode : ASTNode {
    std::string op;
    ASTNode* left;
    ASTNode* right;
};

struct UnaryOpNode : ASTNode {
    std::string op;
    ASTNode* operand;
};

struct VarNode : ASTNode {
    std::string name;
};
```

2.4.2. File ASTBuilder.hpp dan ASTBuilder.cpp

Tabel 2.4.2. Implementasi kelas ASTBuilder

Kelas/Fungsi/Prosedur	**Penjelasan**
class ASTBuilder	Kelas yang bertanggung jawab sebagai Fase 1 dari semantik, di mana struktur ParseNode yang digenerasi oleh modul Parser dikonversi menjadi Abstract Syntax Tree (ASTNode).

<table>
<tr>
<td>

```
class ASTBuilder {
public:
    ASTNode* build(ParseNode* node);

private:
    ASTNode* buildNode(ParseNode* node);
    ASTNode* buildProgram(ParseNode* node);
    ASTNode* buildBlock(ParseNode* node);
    void buildDeclarationPart(ParseNode* node, std::vector<ASTNode*>& decls);
    void buildVarDecl(ParseNode* node, std::vector<ASTNode*>& decls);
    void buildConstDecl(ParseNode* node, std::vector<ASTNode*>& decls);
    void buildTypeDecl(ParseNode* node, std::vector<ASTNode*>& decls);
    ASTNode* buildProcDecl(ParseNode* node);
    ASTNode* buildFuncDecl(ParseNode* node);
    ASTNode* buildType(ParseNode* node);
    ASTNode* buildArrayType(ParseNode* node);
    ASTNode* buildRecordType(ParseNode* node);
    ASTNode* buildRange(ParseNode* node);
    ASTNode* buildEnumerated(ParseNode* node);
    ASTNode* buildStatement(ParseNode* node);
    ASTNode* buildAssign(ParseNode* node);
    ASTNode* buildIf(ParseNode* node);
    ASTNode* buildWhile(ParseNode* node);
    ASTNode* buildFor(ParseNode* node);
    ASTNode* buildRepeat(ParseNode* node);
    ASTNode* buildCase(ParseNode* node);
    ASTNode* buildProcCall(ParseNode* node);
    ASTNode* buildCompound(ParseNode* node);
    ASTNode* buildExpression(ParseNode* node);
    ASTNode* buildSimpleExpression(ParseNode* node);
    ASTNode* buildTerm(ParseNode* node);
    ASTNode* buildFactor(ParseNode* node);
    ASTNode* buildVariable(ParseNode* node);
    ASTNode* buildFuncCall(ParseNode* node);
    ParseNode* findChild(ParseNode* node, const std::string& childName);
    std::vector<ParseNode*> findChildren(ParseNode* node, const std::string& childName);
};
```

</td>
<td></td>
</tr>
<tr>
<td>

ASTNode* ASTBuilder::build(ParseNode* node)

```
ASTNode* ASTBuilder::build(ParseNode* node) {
    return buildNode(node);
}
```

</td>
<td>Entry point untuk kelas builder. Fungsi ini menerima root ParseNode dari hasil sintaksis, memicu rantai pemanggilan konstruksi node, dan mengembalikan root AST.</td>
</tr>
<tr>
<td>

ASTNode* ASTBuilder::buildNode(ParseNode* node)

```
ASTNode* ASTBuilder::buildNode(ParseNode* node) {
    if (!node) return nullptr;
    if (node->name == "<program>")              return buildProgram(node);
    if (node->name == "<block>")                return buildBlock(node);
    if (node->name == "<procedure-declaration>")return buildProcDecl(node);
    if (node->name == "<function-declaration>") return buildFuncDecl(node);
    if (node->name == "<compound-statement>")   return buildCompound(node);
    if (node->name == "<assignment-statement>") return buildAssign(node);
    if (node->name == "<if-statement>")         return buildIf(node);
    if (node->name == "<while-statement>")      return buildWhile(node);
    if (node->name == "<for-statement>")        return buildFor(node);
    if (node->name == "<repeat-statement>")     return buildRepeat(node);
    if (node->name == "<case-statement>")       return buildCase(node);
    if (node->name == "<procedure/function-call>") return buildProcCall(node);
    if (node->name == "<statement>")            return buildStatement(node);
    if (node->name == "<expression>")           return buildExpression(node);
    if (node->name == "<simple-expression>")    return buildSimpleExpression(node);
    if (node->name == "<term>")                 return buildTerm(node);
    if (node->name == "<factor>")               return buildFactor(node);
    if (node->name == "<type>")                 return buildType(node);
    if (node->name == "<variable>")             return buildVariable(node);
    if (node->name == "<array-type>")           return buildArrayType(node);
    if (node->name == "<record-type>")          return buildRecordType(node);
    if (node->name == "<range>")                return buildRange(node);
    if (node->name == "<enumerated>")           return buildEnumerated(node);

    return nullptr;
}
```

</td>
<td>Berperan sebagai dispatcher sentral bottom-up yang membaca nama dari setiap ParseNode dan mendelegasikannya ke fungsi konstruktor individual yang berkaitan. Jika mendeteksi terminal node non-relevan, fungsi mengembalikan nilai nullptr.</td>
</tr>
<tr>
<td>ASTNode* ASTBuilder::buildProgram(ParseNode* node), buildBlock(ParseNode* node), buildProcDecl(ParseNode* node), buildFuncDecl(ParseNode* node),</td>
<td>Kumpulan fungsi pembuat aturan produksi secara spesifik. Fungsi-fungsi ini mengekstrak pointer ParseNode ke level semantik dan mengabaikan token sintaksis</td>
</tr>
</table>

buildType(ParseNode* node), buildArrayType(ParseNode* node), buildRecordType(ParseNode* node), buildRange(ParseNode* node), buildEnumerated(ParseNode* node), buildStatement(ParseNode* node), buildAssign(ParseNode* node), buildIf(ParseNode* node), buildWhile(ParseNode* node), buildFor(ParseNode* node), buildRepeat(ParseNode* node), buildCase(ParseNode* node), buildProcCall(ParseNode* node), buildCompound(ParseNode* node), buildExpression(ParseNode* node), buildSimpleExpression(ParseNode* node), buildTerm(ParseNode* node), buildFactor(ParseNode* node), buildVariable(ParseNode* node), buildFuncCall(ParseNode* node)

```
ASTNode* ASTBuilder::buildProgram(ParseNode* node) {
    auto* p = new ProgramNode();
    p->line = node->line;
    p->kind = ASTKind::Program;
    p->line = node->line;

    ParseNode* header = findChild(node, "<program-header>");
    if (header) {
        ParseNode* ident = findChild(header, "ident");
        if (ident) p->name = ident->value;
    }

    ParseNode* declPart = findChild(node, "<declaration-part>");
    if (declPart) buildDeclarationPart(declPart, p->declarations);

    ParseNode* compound = findChild(node, "<compound-statement>");
    if (compound) p->body = buildCompound(compound);

    return p;
}
```

```
ASTNode* ASTBuilder::buildBlock(ParseNode* node) {
    auto* b = new BlockNode();
    b->line = node->line;
    b->kind = ASTKind::Block;
    b->line = node->line;

    ParseNode* declPart = findChild(node, "<declaration-part>");
    if (declPart) buildDeclarationPart(declPart, b->declarations);

    ParseNode* compound = findChild(node, "<compound-statement>");
    if (compound) b->body = buildCompound(compound);

    return b;
}
```

yang tidak lagi esensial, seperti titik koma atau simbol operasional sintaksis murni.

```
ASTNode* ASTBuilder::buildProcDecl(ParseNode* node) {
    auto* p = new ProcDeclNode();
    p->line = node->line;
    p->kind = ASTKind::ProcDecl;
    p->line = node->line;

    ParseNode* ident = findChild(node, "ident");
    if (ident) p->name = ident->value;

    ParseNode* formals = findChild(node, "<formal-parameter-list>");
    if (formals) {
        for (auto* group : findChildren(formals, "<parameter-group>")) {
            auto* param = new ParamNode();
            param->line = node->line;
            param->kind = ASTKind::Param;
            param->line = group->line;
            param->isByRef = (findChild(group, "varsy") != nullptr);

            ParseNode* idList = findChild(group, "<identifier-list>");
            if (idList) {
                for (auto* id : findChildren(idList, "ident")) {
                    param->names.push_back(id->value);
                }
            }
            ParseNode* t = findChild(group, "<type>");
            if (t) {
                param->typeNode = buildType(t);
            } else {
                for (size_t j = 0; j < group->children.size(); ++j) {
                    if (group->children[j]->name == "colon" && j+1 < group->children.size()) {
                        ParseNode* ti = group->children[j+1];
                        if (ti->name == "ident") {
                            auto* s = new SimpleTypeNode();
                            s->kind = ASTKind::SimpleType; s->typeName = ti->value; s->line = ti->line;
                            param->typeNode = s;
                        }
                        break;
                    }
                }
            }

            p->params.push_back(param);
        }
    }
    ParseNode* block = findChild(node, "<block>");
    if (block) p->block = static_cast<BlockNode*>(buildBlock(block));
    return p;
}
```

```
ASTNode* ASTBuilder::buildFuncDecl(ParseNode* node) {
    auto* f = new FuncDeclNode();
    f->line = node->line;
    f->kind = ASTKind::FuncDecl;
    f->line = node->line;

    ParseNode* ident = findChild(node, "ident");
    if (ident) f->name = ident->value;

    ParseNode* formals = findChild(node, "<formal-parameter-list>");
    if (formals) {
        for (auto* group : findChildren(formals, "<parameter-group>")) {
            auto* param = new ParamNode();
            param->line = node->line;
            param->kind = ASTKind::Param;
            param->line = group->line;
            param->isByRef = (findChild(group, "varsy") != nullptr);

            ParseNode* idList = findChild(group, "<identifier-list>");
            if (idList) {
                for (auto* id : findChildren(idList, "ident")) {
                    param->names.push_back(id->value);
                }
            }
            ParseNode* t = findChild(group, "<type>");
            if (t) {
                param->typeNode = buildType(t);
            } else {
                for (size_t j = 0; j < group->children.size(); ++j) {
                    if (group->children[j]->name == "colon" && j+1 < group->children.size()) {
                        ParseNode* ti = group->children[j+1];
                        if (ti->name == "ident") {
                            auto* s = new SimpleTypeNode();
                            s->kind = ASTKind::SimpleType; s->typeName = ti->value; s->line = ti->line;
                            param->typeNode = s;
                        }
                        break;
                    }
                }
            }
            f->params.push_back(param);
        }
    }

    // Find return type
    for (size_t i = 0; i < node->children.size(); ++i) {
        if (node->children[i]->name == "colon" && i + 1 < node->children.size() && node->children[i+1]->name == "ident") {
            f->returnTypeName = node->children[i+1]->value;
            break;
        }
    }

    ParseNode* block = findChild(node, "<block>");
    if (block) f->block = static_cast<BlockNode*>(buildBlock(block));
    return f;
}
```

```
ASTNode* ASTBuilder::buildType(ParseNode* node) {
    if (node->children.empty()) return nullptr;
    ParseNode* child = node->children[0];
    if (child->name == "ident") {
        auto* s = new SimpleTypeNode();
        s->line = node->line;
        s->kind = ASTKind::SimpleType;
        s->typeName = child->value;
        s->line = child->line;
        return s;
    } else {
        return buildNode(child);
    }
}
```

```
ASTNode* ASTBuilder::buildArrayType(ParseNode* node) {
    auto* a = new ArrayTypeNode();
    a->line = node->line;
    a->kind = ASTKind::ArrayType;
    a->line = node->line;

    ParseNode* indexType = findChild(node, "<range>");
    if (!indexType) indexType = findChild(node, "ident"); // could be simple type

    if (indexType) {
        if (indexType->name == "ident") {
            auto* s = new SimpleTypeNode();
            s->kind = ASTKind::SimpleType;
            s->typeName = indexType->value;
            s->line = indexType->line;
            s->line = node->line;
            a->indexType = s;
        } else {
            a->indexType = buildRange(indexType);
        }
    }

    ParseNode* typeNode = findChild(node, "<type>");
    if (typeNode) a->elementType = buildType(typeNode);
    return a;
}
```

```
ASTNode* ASTBuilder::buildRecordType(ParseNode* node) {
    auto* r = new RecordTypeNode();
    r->line = node->line;
    r->kind = ASTKind::RecordType;
    r->line = node->line;

    ParseNode* fieldList = findChild(node, "<field-list>");
    if (fieldList) {
        for (auto* fieldPart : findChildren(fieldList, "<field-part>")) {
            auto* fd = new FieldDeclNode();
            fd->line = node->line;
            fd->kind = ASTKind::FieldDecl;
            fd->line = fieldPart->line;
            ParseNode* idList = findChild(fieldPart, "<identifier-list>");
            if (idList) {
                for (auto* id : findChildren(idList, "ident")) {
                    fd->names.push_back(id->value);
                }
            }
            ParseNode* t = findChild(fieldPart, "<type>");
            if (t) fd->typeNode = buildType(t);
            r->fields.push_back(fd);
        }
    }
    return r;
}
```

```
ASTNode* ASTBuilder::buildRange(ParseNode* node) {
    auto* r = new RangeNode();
    r->line = node->line;
    r->kind = ASTKind::Range;
    r->line = node->line;

    auto consts = findChildren(node, "<constant>");
    if (consts.size() == 2) {
        // Mock literal construction
        auto makeLit = [](ParseNode* cn) -> ASTNode* {
            if (cn->children.empty()) return nullptr;
            ParseNode* inner = cn->children[0];
            if (inner->name == "intcon") {
                auto* i = new IntLitNode();
                i->kind = ASTKind::IntLit;
                i->value = std::stoi(inner->value);
                return i;
            }
            if (inner->name == "ident") {
                auto* v = new VarNode();
                v->kind = ASTKind::Var;
                v->name = inner->value;
                return v;
            }
            return nullptr;
        };
        r->low = makeLit(consts[0]);
        r->high = makeLit(consts[1]);
    }
    return r;
}
```

```
ASTNode* ASTBuilder::buildEnumerated(ParseNode* node) {
    auto* e = new EnumeratedNode();
    e->line = node->line;
    e->kind = ASTKind::Enumerated;
    e->line = node->line;

    for (auto* id : findChildren(node, "ident")) {
        e->values.push_back(id->value);
    }
    return e;
}

ASTNode* ASTBuilder::buildStatement(ParseNode* node) {
    if (node->children.empty()) return nullptr;
    return buildNode(node->children[0]);
}
```

```
ASTNode* ASTBuilder::buildCompound(ParseNode* node) {
    auto* c = new CompoundNode();
    c->line = node->line;
    c->kind = ASTKind::Compound;
    c->line = node->line;

    ParseNode* stmtList = findChild(node, "<statement-list>");
    if (stmtList) {
        for (auto* stmt : findChildren(stmtList, "<statement>")) {
            ASTNode* st = buildStatement(stmt);
            if (st) {
                c->statements.push_back(st);
            }
        }
    }
    return c;
}
```

```
ASTNode* ASTBuilder::buildAssign(ParseNode* node) {
    auto* a = new AssignNode();
    a->line = node->line;
    a->kind = ASTKind::Assign;
    a->line = node->line;

    a->target = buildVariable(node->children[0]);
    ParseNode* expr = findChild(node, "<expression>");
    if (expr) a->value = buildExpression(expr);

    return a;
}
```

```
ASTNode* ASTBuilder::buildIf(ParseNode* node) {
    auto* i = new IfNode();
    i->line = node->line;
    i->kind = ASTKind::If;
    i->line = node->line;

    ParseNode* exprNode = findChild(node, "<expression>");
    if (exprNode) i->condition = buildExpression(exprNode);
    auto stmts = findChildren(node, "<statement>");
    if (stmts.size() > 0) {
        i->thenBranch = buildStatement(stmts[0]);
    }
    if (stmts.size() > 1) {
        i->elseBranch = buildStatement(stmts[1]);
    }

    return i;
}

ASTNode* ASTBuilder::buildWhile(ParseNode* node) {
    auto* w = new WhileNode();
    w->line = node->line;
    w->kind = ASTKind::While;
    w->line = node->line;

    ParseNode* exprNode = findChild(node, "<expression>");
    if (exprNode) {
        w->condition = buildExpression(exprNode);
    }
    ParseNode* bodyNode = findChild(node, "<statement>");
    if (bodyNode) {
        w->body = buildStatement(bodyNode);
    }

    return w;
}
```

```
ASTNode* ASTBuilder::buildFor(ParseNode* node) {
    auto* f = new ForNode();
    f->line = node->line;
    f->kind = ASTKind::For;
    f->line = node->line;

    ParseNode* ident = findChild(node, "ident");
    if (ident) f->varName = ident->value;

    auto exprs = findChildren(node, "<expression>");
    if (exprs.size() > 0) {
        f->fromExpr = buildExpression(exprs[0]);
    }
    if (exprs.size() > 1) {
        f->toExpr = buildExpression(exprs[1]);
    }

    if (findChild(node, "downtosy")) {
        f->downto = true;
    }

    ParseNode* bodyNode = findChild(node, "<statement>");
    if (bodyNode) {
        f->body = buildStatement(bodyNode);
    }

    return f;
}
```

```
ASTNode* ASTBuilder::buildRepeat(ParseNode* node) {
    auto* r = new RepeatNode();
    r->line = node->line;
    r->kind = ASTKind::Repeat;
    r->line = node->line;

    ParseNode* stmtList = findChild(node, "<statement-list>");
    if (stmtList) {
        for (auto* stmt : findChildren(stmtList, "<statement>")) {
            ASTNode* st = buildStatement(stmt);
            if (st){
                r->body.push_back(st);
            }
        }
    }
    ParseNode* exprNode = findChild(node, "<expression>");
    if (exprNode) {
        r->condition = buildExpression(exprNode);
    }
    return r;
}
```

```
ASTNode* ASTBuilder::buildCase(ParseNode* node) {
    auto* c = new CaseNode();
    c->line = node->line;
    c->kind = ASTKind::Case;
    c->line = node->line;

    ParseNode* exprNode = findChild(node, "<expression>");
    if (exprNode) c->expr = buildExpression(exprNode);

    std::vector<ParseNode*> allBlocks;
    ParseNode* firstBlock = findChild(node, "<case-block>");
    flattenCaseBlocks(firstBlock, allBlocks);

    for (auto* block : allBlocks) {
        auto* branch = new CaseBranchNode();
        branch->line = node->line;
        branch->kind = ASTKind::CaseBranch;
        branch->line = block->line;

        for (auto* cnst : findChildren(block, "<constant>")) {
            // Very simplified
            if (!cnst->children.empty()) {
                ParseNode* inner = cnst->children[0];
                if (inner->name == "intcon") {
                    auto* i = new IntLitNode();
                    i->kind = ASTKind::IntLit;
                    i->value = std::stoi(inner->value);
                    branch->labels.push_back(i);
                }
                else if (inner->name == "ident") {
                    auto* v = new VarNode();
                    v->kind = ASTKind::Var;
                    v->name = inner->value;
                    branch->labels.push_back(v);
                }
            }
        }
        branch->statement = buildStatement(findChild(block, "<statement>"));
        c->branches.push_back(branch);
    }
    return c;
}
```

```
ASTNode* ASTBuilder::buildProcCall(ParseNode* node) {
    auto* p = new ProcCallNode();
    p->line = node->line;
    p->kind = ASTKind::ProcCall;
    p->line = node->line;

    ParseNode* ident = findChild(node, "ident");
    if (ident) p->name = ident->value;

    ParseNode* params = findChild(node, "<parameter-list>");
    if (params) {
        for (auto* expr : findChildren(params, "<expression>")) {
            p->args.push_back(buildExpression(expr));
        }
    }
    return p;
}

ASTNode* ASTBuilder::buildFuncCall(ParseNode* node) {
    auto* f = new FuncCallNode();
    f->line = node->line;
    f->kind = ASTKind::FuncCall;
    f->line = node->line;

    ParseNode* ident = findChild(node, "ident");
    if (ident) f->name = ident->value;

    ParseNode* params = findChild(node, "<parameter-list>");
    if (params) {
        for (auto* expr : findChildren(params, "<expression>")) {
            f->args.push_back(buildExpression(expr));
        }
    }
    return f;
}
```

```cpp
// Flat tree generation for binary expressions
ASTNode* ASTBuilder::buildExpression(ParseNode* node) {
    auto& children = node->children;
    ASTNode* left = nullptr;

    for (size_t i = 0; i < children.size(); i++) {
        if (children[i]->name == "<simple-expression>") {
            if (!left) {
                left = buildSimpleExpression(children[i]);
            } else {
                std::string op = tokenNameToOp(children[i-1]->children[0]->name);
                auto* bin = new BinOpNode();
                bin->line = node->line;
                bin->kind = ASTKind::BinOp;
                bin->line = children[i-1]->line;
                bin->op = op;
                bin->left = left;
                bin->right = buildSimpleExpression(children[i]);
                left = bin;
            }
        }
    }
    return left;
}
```

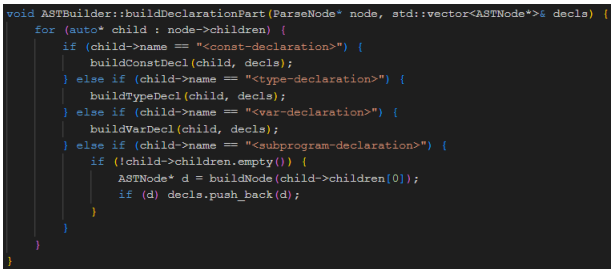
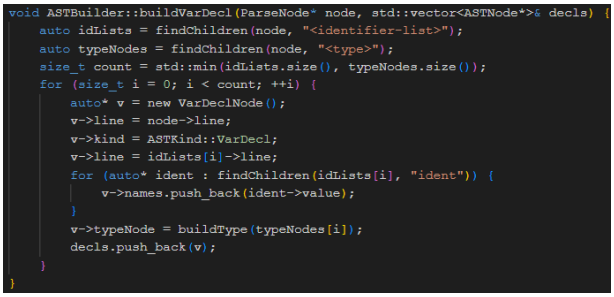
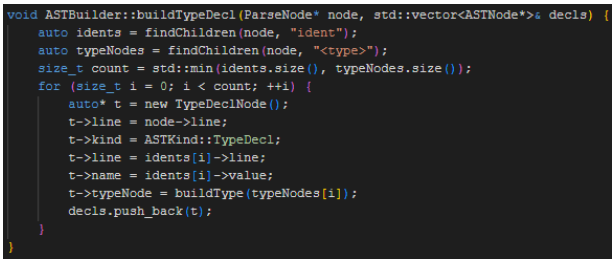
```cpp
ASTNode* ASTBuilder::buildSimpleExpression(ParseNode* node) {
    auto& children = node->children;
    ASTNode* left = nullptr;

    size_t i = 0;
    if (i < children.size() && (children[i]->name == "plus" || children[i]->name == "minus")) {
        auto* un = new UnaryOpNode();
        un->line = node->line;
        un->kind = ASTKind::UnaryOp;
        un->line = children[i]->line;
        un->op = tokenNameToOp(children[i]->name);
        i++;
        if (i < children.size() && children[i]->name == "<term>") {
            un->operand = buildTerm(children[i]);
            left = un;
            i++;
        }
    }

    for (; i < children.size(); i++) {
        if (children[i]->name == "<term>") {
            if (!left) {
                left = buildTerm(children[i]);
            } else {
                std::string op = tokenNameToOp(children[i-1]->children[0]->name);
                auto* bin = new BinOpNode();
                bin->line = node->line;
                bin->kind = ASTKind::BinOp;
                bin->line = children[i-1]->line;
                bin->op = op;
                bin->left = left;
                bin->right = buildTerm(children[i]);
                left = bin;
            }
        }
    }
    return left;
}
```

```cpp
ASTNode* ASTBuilder::buildTerm(ParseNode* node) {
    auto& children = node->children;
    ASTNode* left = nullptr;

    for (size_t i = 0; i < children.size(); i++) {
        if (children[i]->name == "<factor>") {
            if (!left) {
                left = buildFactor(children[i]);
            } else {
                std::string op = tokenNameToOp(children[i-1]->children[0]->name);
                auto* bin = new BinOpNode();
                bin->line = node->line;
                bin->kind = ASTKind::BinOp;
                bin->line = children[i-1]->line;
                bin->op = op;
                bin->left = left;
                bin->right = buildFactor(children[i]);
                left = bin;
            }
        }
    }
    return left;
}
```

```
ASTNode* ASTBuilder::buildFactor(ParseNode* node) {
    if (node->children.empty()) return nullptr;
    ParseNode* child = node->children[0];

    if (child->name == "<procedure/function-call>") {
        return buildFuncCall(child);
    }
    if (child->name == "<variable>") {
        return buildVariable(child);
    }
    if (child->name == "intcon")  {
        auto* i = new IntLitNode();
        i->kind = ASTKind::IntLit;
        i->value = std::stoi(child->value);
        i->line = child->line;
        return i;
    }
    if (child->name == "realcon") {
        auto* r = new RealLitNode();
        r->kind = ASTKind::RealLit;
        r->value = std::stod(child->value);
        r->line = child->line;
        return r;
    }
    if (child->name == "charcon") {
        auto* c = new CharLitNode();
        c->kind = ASTKind::CharLit;
        c->value = child->value.size()>=2 ? child->value[1] : '\0';
        c->line = child->line; return c; }
    if (child->name == "string")  {
        auto* s = new StringLitNode();
        s->kind = ASTKind::StringLit;
        s->value = child->value;
        s->line = child->line;
        return s; }
    if (child->name == "boolcon") {
        auto* b = new BoolLitNode();
        b->kind = ASTKind::BoolLit;
        b->value = (child->value == "true");
        b->line = child->line;
        return b; }
    if (child->name == "notsy")   {
        auto* u = new UnaryOpNode();
        u->kind = ASTKind::UnaryOp;
        u->op = "not";
        u->line = child->line;
        u->operand = buildFactor(node->children[1]);
        return u; }
    if (child->name == "lparent") {
        return buildExpression(node->children[1]);
    }

    return nullptr;
}
```

```
ASTNode* ASTBuilder::buildVariable(ParseNode* node) {
    ParseNode* ident = findChild(node, "ident");
    if (!ident) return nullptr;

    ASTNode* base = new VarNode();
    base->kind = ASTKind::Var;
    static_cast<VarNode*>(base)->name = ident->value;
    base->line = ident->line;

    for (auto* comp : findChildren(node, "<component-variable>")) {
        if (comp->children.empty()) continue;
        if (comp->children[0]->name == "lbrack") {
            ParseNode* idxList = findChild(comp, "<index-list>");
            if (idxList) {
                // assume single index for now
                if (idxList && !idxList->children.empty()) {
                    ParseNode* idxTok = idxList->children[0];  // intcon, charcon, atau ident
                    ASTNode* idxNode = nullptr;
                    if (idxTok->name == "intcon") {
                        auto* i = new IntLitNode(); i->kind = ASTKind::IntLit;
                        i->value = std::stoi(idxTok->value); i->line = idxTok->line;
                        idxNode = i;
                    } else if (idxTok->name == "ident") {
                        auto* v = new VarNode(); v->kind = ASTKind::Var;
                        v->name = idxTok->value; v->line = idxTok->line;
                        idxNode = v;
                    } else if (idxTok->name == "charcon") {
                        auto* c = new CharLitNode(); c->kind = ASTKind::CharLit;
                        c->value = idxTok->value.size() >= 2 ? idxTok->value[1] : '\0'; c->line = idxTok->line;
                        idxNode = c;
                    }
                    if (idxNode) {
                        auto* arr = new ArrayAccessNode();
                        arr->kind = ASTKind::ArrayAccess; arr->line = comp->line;
                        arr->array = base; arr->index = idxNode;
                        base = arr;
                    }
                }
            }
        } else if (comp->children[0]->name == "period") {
            ParseNode* field = findChild(comp, "ident");
            if (field) {
                auto* rec = new RecordAccessNode();
                rec->line = node->line;
                rec->kind = ASTKind::RecordAccess;
                rec->line = comp->line;
                rec->record = base;
                rec->field = field->value;
                base = rec;
            }
        }
    }
    return base;
}
```

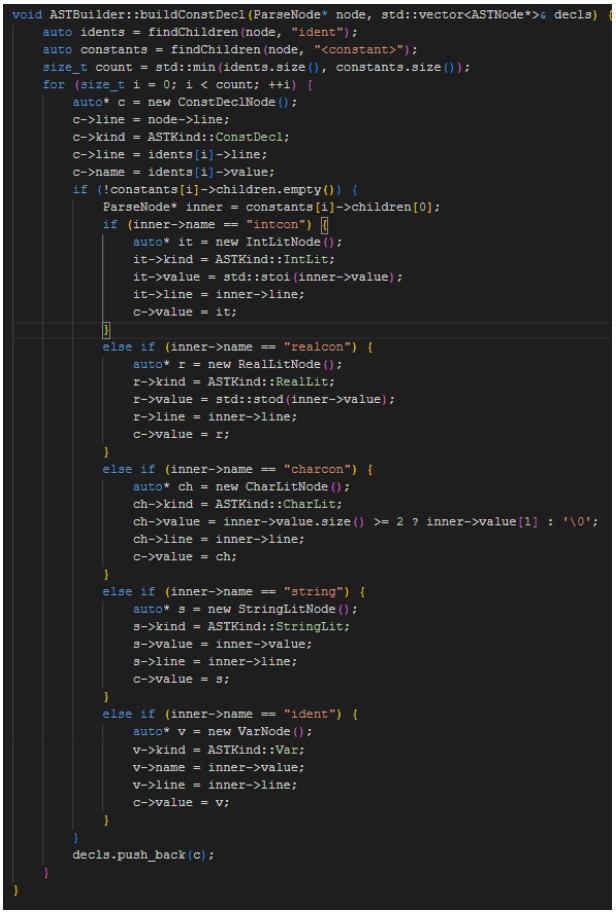
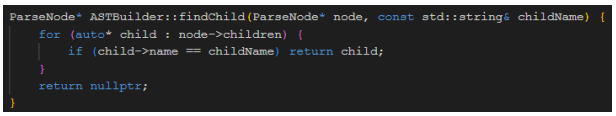
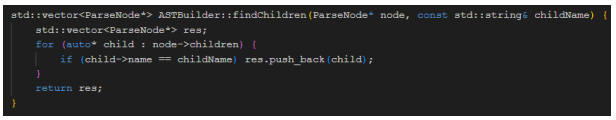
void ASTBuilder::buildDeclarationPart(ParseNode*	Menangani sekumpulan deklarasi yang

node, std::vector<ASTNode*>& decls), buildVarDecl(ParseNode* node, std::vector<ASTNode*>& decls), buildTypeDecl(ParseNode* node, std::vector<ASTNode*>& decls), buildConstDecl(ParseNode* node, std::vector<ASTNode*>& decls)   	berdekatan atau bersarang dan memasukkan hasil pembuatannya secara langsung ke dalam wadah milik *node* induknya.

```
void ASTBuilder::buildDeclarationPart(ParseNode* node, std::vector<ASTNode*>& decls) {
    for (auto* child : node->children) {
        if (child->name == "<const-declaration>") {
            buildConstDecl(child, decls);
        } else if (child->name == "<type-declaration>") {
            buildTypeDecl(child, decls);
        } else if (child->name == "<var-declaration>") {
            buildVarDecl(child, decls);
        } else if (child->name == "<subprogram-declaration>") {
            if (!child->children.empty()) {
                ASTNode* d = buildNode(child->children[0]);
                if (d) decls.push_back(d);
            }
        }
    }
}
```

```
void ASTBuilder::buildVarDecl(ParseNode* node, std::vector<ASTNode*>& decls) {
    auto idLists = findChildren(node, "<identifier-list>");
    auto typeNodes = findChildren(node, "<type>");
    size_t count = std::min(idLists.size(), typeNodes.size());
    for (size_t i = 0; i < count; ++i) {
        auto* v = new VarDeclNode();
        v->line = node->line;
        v->kind = ASTKind::VarDecl;
        v->line = idLists[i]->line;
        for (auto* ident : findChildren(idLists[i], "ident")) {
            v->names.push_back(ident->value);
        }
        v->typeNode = buildType(typeNodes[i]);
        decls.push_back(v);
    }
}
```

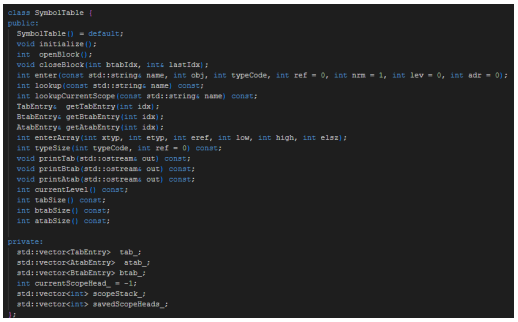
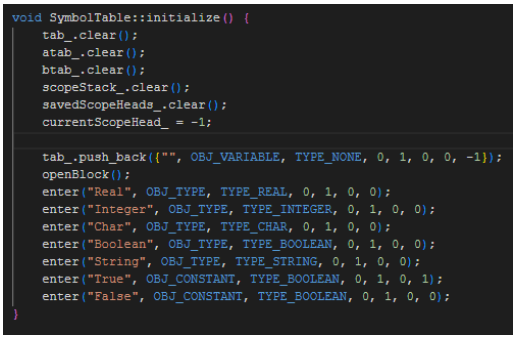
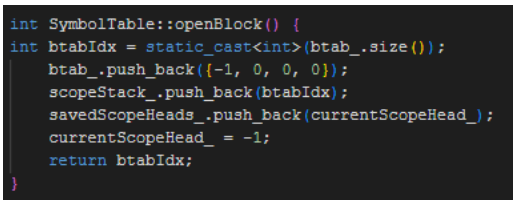
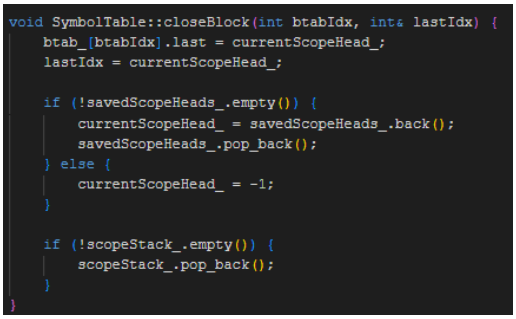
```
void ASTBuilder::buildTypeDecl(ParseNode* node, std::vector<ASTNode*>& decls) {
    auto idents = findChildren(node, "ident");
    auto typeNodes = findChildren(node, "<type>");
    size_t count = std::min(idents.size(), typeNodes.size());
    for (size_t i = 0; i < count; ++i) {
        auto* t = new TypeDeclNode();
        t->line = node->line;
        t->kind = ASTKind::TypeDecl;
        t->line = idents[i]->line;
        t->name = idents[i]->value;
        t->typeNode = buildType(typeNodes[i]);
        decls.push_back(t);
    }
}
```

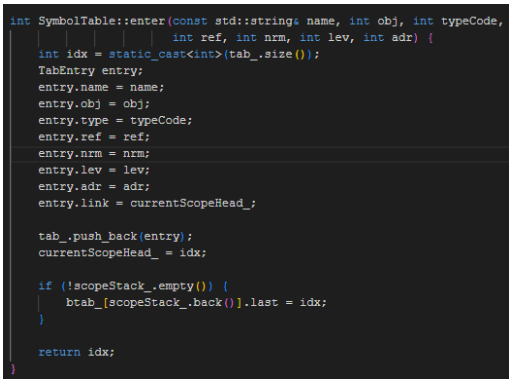
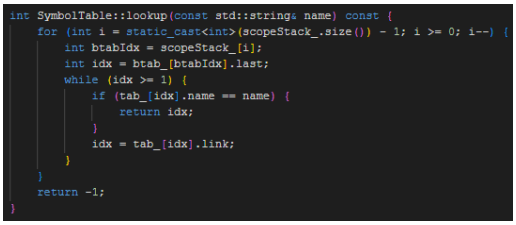
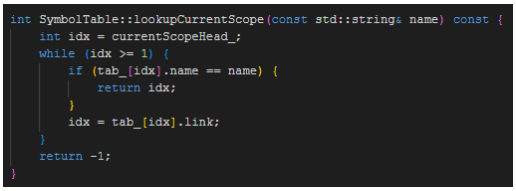
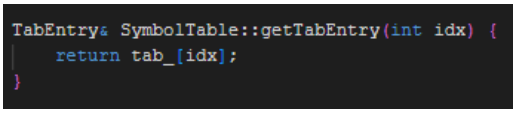
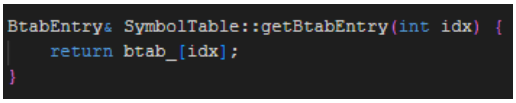
	
ParseNode* ASTBuilder::findChild(ParseNode* node, const std::string& childName)  std::vector<ParseNode*> ASTBuilder::findChildren(ParseNode* node, const std::string& childName) 	Fungsi ini tidak membangun *node* AST sama sekali. Tugas murninya adalah sebagai alat bantu navigasi pohon untuk mencari letak *node* sintaksis tertentu.

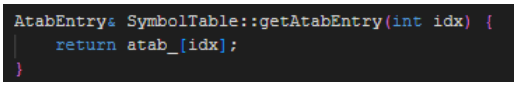
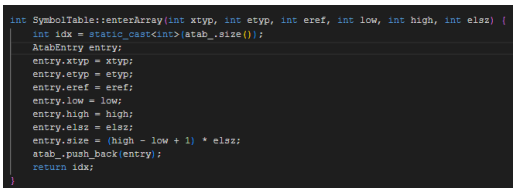
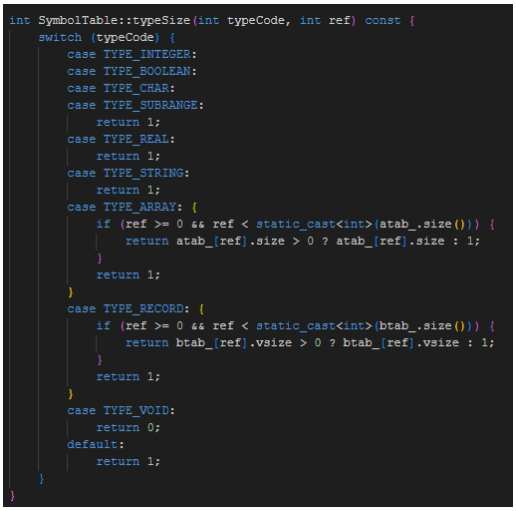
2.4.3. File SymbolTable.hpp dan SymbolTable.cpp

Tabel 2.4.3. Implementasi kelas SymbolTable

Kelas/Fungsi/Prosedur	**Penjelasan**
struct TabEntry `struct TabEntry { std::string name; int obj = OBJ_VARIABLE; int type = TYPE_NONE; int ref = 0; int nrm = 1; int lev = 0; int adr = 0; int link = -1; };`	Struktur data yang merepresentasikan satu entri simbol dalam tabel simbol utama (tab). Setiap entri menyimpan nama simbol, kategori objek (OBJ_VARIABLE, OBJ_CONSTANT, OBJ_TYPE, OBJ_PROCEDURE, OBJ_FUNCTION, atau OBJ_PROGRAM), kode tipe data, indeks referensi ke atab atau btab (ref), flag parameter pass-by-value (nrm), tingkat leksikal (lev), alamat offset (adr), serta pointer linked list ke entri sebelumnya dalam scope yang sama (link).
struct ATabEntry `struct AtabEntry { int xtyp = TYPE_INTEGER; int etyp = TYPE_NONE; int eref = 0; int low = 0; int high = 0; int elsz = 1; int size = 0; };`	Struktur data yang merepresentasikan entri tipe array dalam tabel array (atab). Menyimpan informasi tipe indeks (xtyp), tipe elemen (etyp), referensi elemen jika berupa array bersarang (eref), batas bawah (low) dan batas atas (high) dari rentang indeks, ukuran tiap elemen dalam satuan slot memori (elsz), serta ukuran total array (size = (high - low + 1) * elsz).
struct BtabEntry `struct BtabEntry { int last = -1; int lpar = 0; int psize = 0; int vsize = 0; };`	Struktur data yang merepresentasikan satu blok scope dalam tabel blok (btab), yang digunakan untuk prosedur, fungsi, maupun record. Menyimpan indeks entri simbol terakhir yang terdaftar di blok tersebut (last), indeks parameter formal terakhir (lpar), ukuran area parameter dalam satuan slot (psize), dan ukuran total area variabel lokal (vsize).
class SymbolTable	Kelas utama yang mengelola ketiga tabel internal: tab_ (tabel simbol), atab_ (tabel array), dan btab_ (tabel blok). Mengimplementasikan mekanisme scope bertingkat berbasis scopeStack_ serta linked list per scope untuk mendukung lookup hierarkis dari scope

	terdalam ke terluar. Instansi tunggal kelas ini dibagikan ke seluruh komponen analisis semantik.
void SymbolTable::initialize() 	Menginisialisasi ulang seluruh tabel internal ke kondisi kosong, lalu membuka blok scope level-0 dan mendaftarkan seluruh identifier bawaan bahasa Arion: tipe primitif Real, Integer, Char, Boolean, dan String (sebagai OBJ_TYPE), serta konstanta True dan False (sebagai OBJ_CONSTANT).
int SymbolTable::openBlock() 	Membuka scope atau blok baru dengan mendorong BtabEntry baru ke btab_, menyimpan currentScopeHead_ ke savedScopeHeads_, dan mereset currentScopeHead_ ke -1. Mengembalikan indeks btab dari blok yang baru dibuka; indeks ini digunakan untuk menutup blok dan menghubungkan entri prosedur atau fungsi ke bloknya.
void SymbolTable::closeBlock(int btabIdx, int& lastIdx) 	Menutup scope aktif dengan menyimpan currentScopeHead_ ke btab_[btabIdx].last, merestorasi head scope sebelumnya dari savedScopeHeads_, serta memperbarui parameter lastIdx sebagai referensi ke entri simbol terakhir di blok tersebut. Dipanggil setelah seluruh deklarasi dalam satu blok selesai diproses.
int SymbolTable::enter(const std::string&	Mendaftarkan entri simbol baru ke tabel simbol.

name, int obj, int typeCode, int ref, int nrm, int lev, int adr)  	Membuat TabEntry dengan seluruh atribut yang diberikan, menetapkan link ke currentScopeHead_ (linked list scope), memperbarui currentScopeHead_ ke indeks entri baru, dan memperbaharui btab_ scope aktif. Mengembalikan indeks entri yang baru dibuat.
int SymbolTable::lookup(const std::string& name) const  	Mencari simbol berdasarkan nama secara hierarkis mulai dari scope terdalam ke scope terluar menggunakan iterasi pada scopeStack_ dan traversal linked list tiap scope. Mengembalikan indeks entri pada tab_ jika ditemukan, atau -1 jika tidak ada simbol yang cocok di seluruh rantai scope.
int SymbolTable::lookupCurrentScope(const std::string& name) const  	Mencari simbol hanya pada scope aktif saat ini menggunakan traversal currentScopeHead_ melalui linked list. Digunakan khusus untuk mendeteksi redeklarasi identifier dalam satu scope yang sama, sehingga deklarasi ulang dengan nama yang sama dilaporkan sebagai kesalahan semantik.
TabEntry& getTabEntry(int idx),  BtabEntry& getBtabEntry(int idx),  AtabEntry& getAtabEntry(int idx)	Kumpulan fungsi getter yang menyediakan akses referensi langsung (by reference) ke entri pada masing-masing tabel internal berdasarkan indeks integer. Digunakan secara luas oleh komponen analisis semantik untuk membaca atribut simbol maupun memperbarui field seperti ref pada entri prosedur atau fungsi setelah blok scope-nya diketahui.

	
int SymbolTable::enterArray(int xtyp, int etyp, int eref, int low, int high, int elsz) 	Mendaftarkan entri tipe array baru ke atab_ dengan menghitung ukuran total array secara otomatis: size = (high - low + 1) * elsz. Mengembalikan indeks atab yang kemudian disimpan sebagai field ref pada entri variabel atau tipe bertipe array.
int SymbolTable::typeSize(int typeCode, int ref) const 	Menghitung dan mengembalikan ukuran memori dalam satuan slot dari suatu tipe data. Tipe skalar Integer, Real, Boolean, Char, dan String masing-masing bernilai 1. Untuk TYPE_ARRAY, ukuran diperoleh dari atab_[ref].size, sedangkan untuk TYPE_RECORD diperoleh dari btab_[ref].vsize. TYPE_VOID mengembalikan 0.

2.4.4. File SemanticAnalyzer.hpp dan SemanticAnalyzer.cpp

Tabel 2.4.4. Implementasi kelas header SemanticAnalyzer

Kelas/Fungsi/Prosedur	**Penjelasan**
class SemanticAnalyzer	Kelas yang berfungsi untuk menganalisis semantik Fase 2 dengan mengimplementasikan pola arsitektur Semantic Visitor DFS. Bertugas memberikan anotasi tipe data, jangkauan ruang lingkup (scope), dan integrasi Tabel Simbol secara *in-place*.

<table>
<tr><td>

```
class SemanticAnalyzer {
public:
    SemanticAnalyzer();
    ~SemanticAnalyzer() = default;

    void analyze(ASTNode* root);
    bool hasError() const;
    void printSymbolTables(std::ostream& out) const;
    void visitProgram(ProgramNode* node);
    void visitBlock(BlockNode* node);
    void visitVarDecl(VarDeclNode* node);
    void visitConstDecl(ConstDeclNode* node);
    void visitTypeDecl(TypeDeclNode* node);
    void visitProcDecl(ProcDeclNode* node);
    void visitFuncDecl(FuncDeclNode* node);
    void visitCompound(CompoundNode* node);
    void visitAssign(AssignNode* node);
    void visitIf(IfNode* node);
    void visitWhile(WhileNode* node);
    void visitFor(ForNode* node);
    void visitRepeat(RepeatNode* node);
    void visitCase(CaseNode* node);
    void visitProcCall(ProcCallNode* node);
    void visitExpr(ASTNode* node);
    void visitBinOp(BinOpNode* node);
    void visitUnaryOp(UnaryOpNode* node);
    void visitVar(VarNode* node);
    void visitArrayAccess(ArrayAccessNode* node);
    void visitRecordAccess(RecordAccessNode* node);
    void visitFuncCall(FuncCallNode* node);
    void visitIntLit(IntLitNode* node);
    void visitRealLit(RealLitNode* node);
    void visitCharLit(CharLitNode* node);
    void visitStringLit(StringLitNode* node);
    void visitBoolLit(BoolLitNode* node);

private:
    ms3::SymbolTable symTable;
    TypeChecker typeChecker;
    std::vector<std::string> errors_;
    int currentLevel_ = 0;

    void semanticError(const std::string& msg, int line);
    void visitNode(ASTNode* node);
};
```

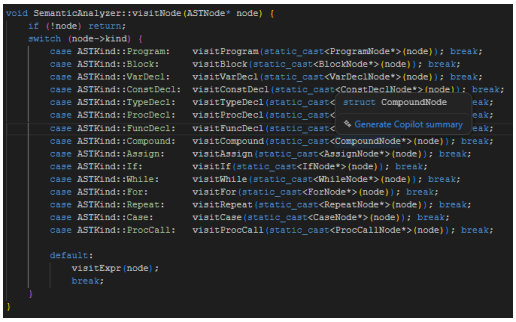
</td><td></td></tr>
<tr><td>SemanticAnalyzer::SemanticAnalyzer()

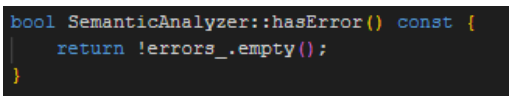
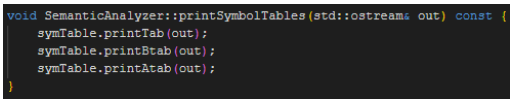
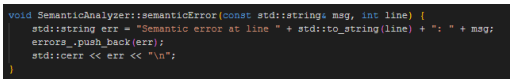
```
SemanticAnalyzer::SemanticAnalyzer() {
    symTable.initialize();
}
```

</td><td>Konstruktor yang menyiapkan state analisis dan menginisialisasi pustaka identifier predefined atau tipe bawaan bahasa langsung ke dalam SymbolTable saat instansiasi kelas.</td></tr>
<tr><td>void SemanticAnalyzer::analyze(ASTNode* root)

```
void SemanticAnalyzer::analyze(ASTNode* root) {
    if (!root) return;
    visitNode(root);
}
```

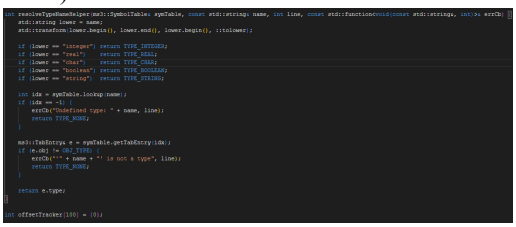
</td><td>Entry point utama fase anotasi. Memvalidasi bahwa root tidak kosong, kemudian memicu proses evaluasi rekursif pada node teratas dengan pemanggilan visitNode().</td></tr>
<tr><td>void SemanticAnalyzer::visitNode(ASTNode* node)</td><td>Dispatcher visitor utama yang bertugas untuk mengevaluasi kind dari setiap node pohon yang masuk menggunakan struktur switch-case, untuk memanggil sub-visitor spesifik yang menangani bentuk node tersebut.</td></tr>
</table>

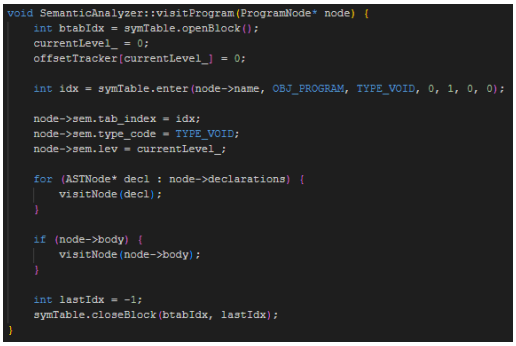
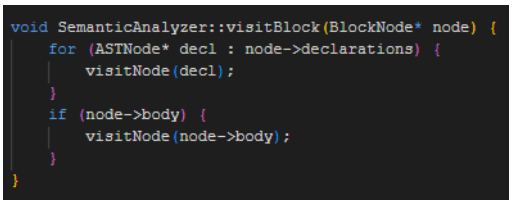
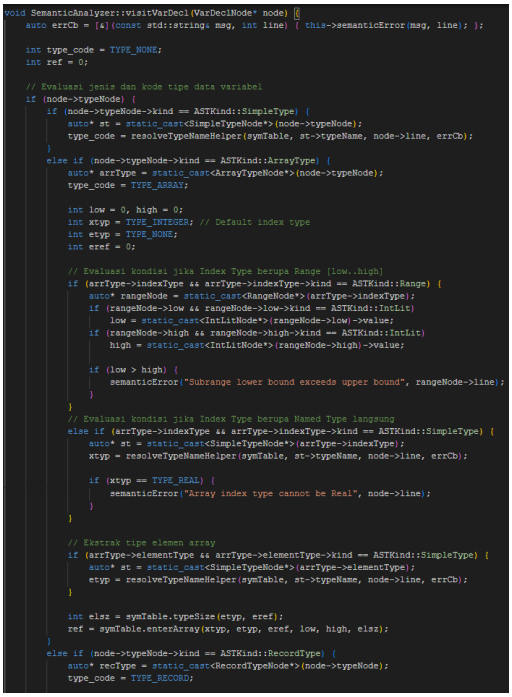
 	
void SemanticAnalyzer::visitProgram(ProgramNode* node), visitBlock(BlockNode* node), visitVarDecl(VarDeclNode* node), visitConstDecl(ConstDeclNode* node), visitTypeDecl(TypeDeclNode* node), visitProcDecl(ProcDeclNode* node), visitFuncDecl(FuncDeclNode* node), visitCompound(CompoundNode* node), visitAssign(AssignNode* node), visitIf(IfNode* node), visitWhile(WhileNode* node), visitFor(ForNode* node), visitRepeat(RepeatNode* node), visitCase(CaseNode* node), visitProcCall(ProcCallNode* node), visitExpr(ASTNode* node), visitBinOp(BinOpNode* node), visitUnaryOp(UnaryOpNode* node), visitVar(VarNode* node), visitArrayAccess(ArrayAccessNode* node), visitRecordAccess(RecordAccessNode* node), visitFuncCall(FuncCallNode* node), visitIntLit(IntLitNode* node), visitRealLit(RealLitNode* node), visitCharLit(CharLitNode* node), visitStringLit(StringLitNode* node), visitBoolLit(BoolLitNode* node)	Kumpulan method spesifik yang bertugas melakukan iterasi semantik, memeriksa integritas tipe (type checking), dan mendeteksi kesalahan leksikal dalam tingkat yang lebih kecil.
bool SemanticAnalyzer::hasError() const	Fungsi pengecekan status akhir analisis semantik.

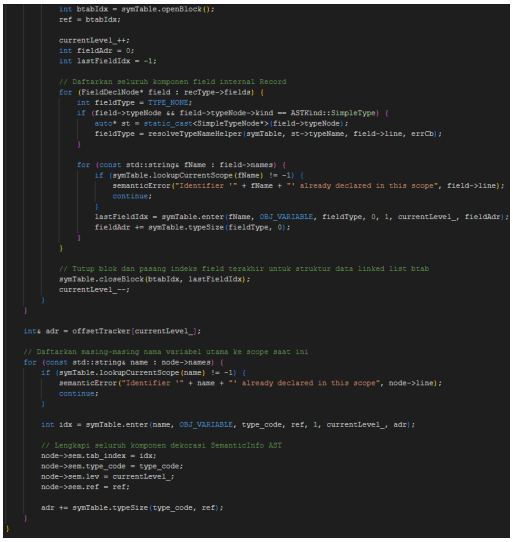
	Mengembalikan true jika daftar errors_ tidak kosong, yaitu apabila setidaknya satu kesalahan semantik telah dicatat selama proses analisis. Dipanggil oleh main() untuk menentukan kode keluar (exit code) program.
void SemanticAnalyzer::printSymbolTables(std::ostream& out) const 	Fungsi utilitas untuk mencetak ketiga tabel internal SymbolTable (tab, btab, dan atab) ke stream output yang diberikan. Memanggil printTab(), printBtab(), dan printAtab() secara berurutan. Digunakan pada akhir proses analisis untuk menghasilkan tampilan debug tabel simbol yang terstruktur.
void SemanticAnalyzer::semanticError(const std::string& msg, int line) 	Fungsi privat untuk mencatat dan menampilkan kesalahan semantik. Memformat pesan dengan menyertakan nomor baris sumber, mendorongnya ke vektor errors_, sekaligus menulisnya langsung ke stderr. Dipanggil oleh seluruh visitor kapanpun ditemukan pelanggaran aturan semantik.

2.4.5. File visit_declarations.cpp

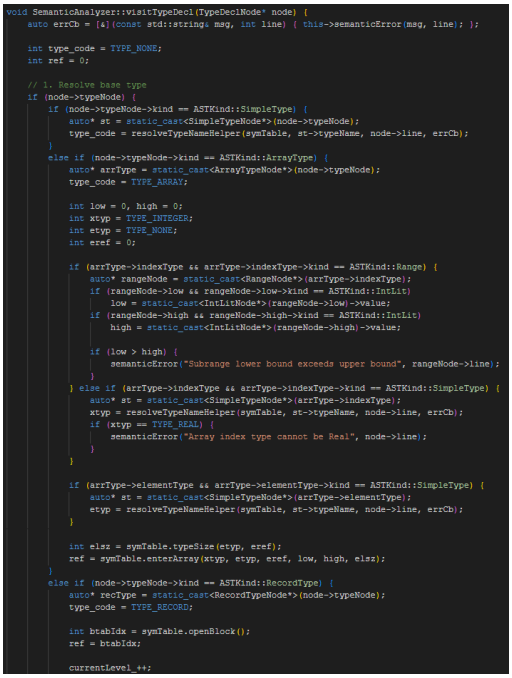
Tabel 2.4.5. Implementasi kelas visit_declarations

Kelas/Fungsi/Prosedur	**Penjelasan**
int resolveTypeNameHelper (ms3::SymbolTable& symTable, const std::string& name, int line, const std::function<void(const std::string&, int)>& errCb) 	Fungsi helper yang digunakan untuk mengonversi nama tipe data berupa string menjadi konstanta numerik yang ekuivalen. Jika tipe tidak dikenali sebagai tipe dasar, fungsi ini akan melakukan *lookup* ke symbol table untuk memastikan apakah string tersebut adalah *user-defined type* yang valid.
SemanticAnalyzer::visitProgram(ProgramNode * node)	Visitor untuk node program (root AST). Membuka blok scope level-0, mendaftarkan nama program ke tabel simbol sebagai OBJ_PROGRAM bertipe

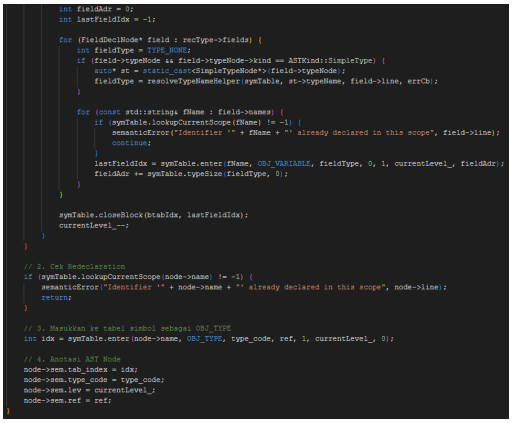
	TYPE_VOID, menjalankan visitor untuk seluruh deklarasi global di dalam node, kemudian mengunjungi body program (CompoundNode), dan menutup blok scope di akhir proses.
SemanticAnalyzer::visitBlock(BlockNode* node) 	Visitor untuk node blok yang memproses bagian declarations dan body secara berurutan. Dipanggil oleh visitProcDecl dan visitFuncDecl untuk menangani deklarasi-deklarasi lokal serta statement body di dalam scope prosedur atau fungsi yang sudah dibuka sebelumnya.
SemanticAnalyzer::visitVarDecl(VarDeclNode* node) 	Visitor untuk deklarasi variabel. Menentukan kode tipe data berdasarkan jenis typeNode (SimpleType, ArrayType, atau RecordType). Untuk ArrayType, mendaftarkan entri ke atab dan menghitung ukurannya; untuk RecordType, membuka blok btab tersendiri dan mendaftarkan seluruh field-nya. Setiap nama variabel kemudian didaftarkan ke scope aktif dengan alamat offset yang dihitung bertahap, disertai pengecekan redeklarasi.

	
SemanticAnalyzer::visitConstDecl(ConstDeclNode* node) 	Visitor untuk deklarasi konstanta. Mengevaluasi tipe dan nilai primitif dari ekspresi literal (IntLit, CharLit, BoolLit), lalu mendaftarkan nama konstanta ke tabel simbol sebagai OBJ_CONSTANT dengan nilai literal disimpan pada field adr. Redeklarasi nama yang sudah ada di scope saat ini dilaporkan sebagai kesalahan.
SemanticAnalyzer::visitTypeDecl(TypeDeclNode* node)	Visitor untuk deklarasi tipe yang ditentukan pengguna. Mengevaluasi definisi tipe (SimpleType, ArrayType, atau RecordType) dengan logika yang identik dengan visitVarDecl, kemudian mendaftarkan nama alias tipe baru ke tabel simbol sebagai OBJ_TYPE dengan kode tipe dan referensi yang sesuai.

<table>
<tr>
<td>



```
void SemanticAnalyzer::visitTypeDecl(TypeDeclNode* node) {
    auto errCb = [&](const std::string& msg, int line) { this->semanticError(msg, line); };

    int type_code = TYPE_NONE;
    int ref = 0;

    // 1. Resolve base type
    if (node->typeNode) {
        if (node->typeNode->kind == ASTKind::SimpleType) {
            auto* st = static_cast<SimpleTypeNode*>(node->typeNode);
            type_code = resolveTypeNameHelper(symTable, st->typeName, node->line, errCb);
        }
        else if (node->typeNode->kind == ASTKind::ArrayType) {
            auto* arrType = static_cast<ArrayTypeNode*>(node->typeNode);
            type_code = TYPE_ARRAY;

            int low = 0, high = 0;
            int xtyp = TYPE_INTEGER;
            int etyp = TYPE_NONE;
            int eref = 0;

            if (arrType->indexType && arrType->indexType->kind == ASTKind::Range) {
                auto* rangeNode = static_cast<RangeNode*>(arrType->indexType);
                if (rangeNode->low && rangeNode->low->kind == ASTKind::IntLit)
                    low = static_cast<IntLitNode*>(rangeNode->low)->value;
                if (rangeNode->high && rangeNode->high->kind == ASTKind::IntLit)
                    high = static_cast<IntLitNode*>(rangeNode->high)->value;

                if (low > high) {
                    semanticError("Subrange lower bound exceeds upper bound", rangeNode->line);
                }
            } else if (arrType->indexType && arrType->indexType->kind == ASTKind::SimpleType) {
                auto* st = static_cast<SimpleTypeNode*>(arrType->indexType);
                xtyp = resolveTypeNameHelper(symTable, st->typeName, node->line, errCb);
                if (xtyp == TYPE_REAL) {
                    semanticError("Array index type cannot be Real", node->line);
                }
            }

            if (arrType->elementType && arrType->elementType->kind == ASTKind::SimpleType) {
                auto* st = static_cast<SimpleTypeNode*>(arrType->elementType);
                etyp = resolveTypeNameHelper(symTable, st->typeName, node->line, errCb);
            }

            int elsz = symTable.typeSize(etyp, eref);
            ref = symTable.enterArray(xtyp, etyp, eref, low, high, elsz);
        }
        else if (node->typeNode->kind == ASTKind::RecordType) {
            auto* recType = static_cast<RecordTypeNode*>(node->typeNode);
            type_code = TYPE_RECORD;

            int btabIdx = symTable.openBlock();
            ref = btabIdx;

            currentLevel_++;
```



```
            int fieldAddr = 0;
            int lastFieldIdx = -1;

            for (FieldDeclNode* field : recType->fields) {
                int fieldType = TYPE_NONE;
                if (field->typeNode && field->typeNode->kind == ASTKind::SimpleType) {
                    auto* st = static_cast<SimpleTypeNode*>(field->typeNode);
                    fieldType = resolveTypeNameHelper(symTable, st->typeName, field->line, errCb);
                }

                for (const std::string& fName : field->names) {
                    if (symTable.lookupCurrentScope(fName) != -1) {
                        semanticError("Identifier '" + fName + "' already declared in this scope", field->line);
                        continue;
                    }
                    lastFieldIdx = symTable.enter(fName, OBJ_VARIABLE, fieldType, 0, 1, currentLevel_, fieldAddr);
                    fieldAddr += symTable.typeSize(fieldType, 0);
                }
            }

            symTable.closeBlock(btabIdx, lastFieldIdx);
            currentLevel_--;
        }
    }

    // 2. Cek Redeclaration
    if (symTable.lookupCurrentScope(node->name) != -1) {
        semanticError("Identifier '" + node->name + "' already declared in this scope", node->line);
        return;
    }

    // 3. Masukkan ke tabel simbol sebagai OBJ_TYPE
    int idx = symTable.enter(node->name, OBJ_TYPE, type_code, ref, 1, currentLevel_, 0);

    // 4. Anotasi AST Node
    node->sem.tab_index = idx;
    node->sem.type_code = type_code;
    node->sem.lev = currentLevel_;
    node->sem.ref = ref;
}
```

</td>
<td></td>
</tr>
<tr>
<td>SemanticAnalyzer::visitProcDecl(ProcDeclNode* node)</td>
<td>Visitor untuk deklarasi prosedur. Mendaftarkan nama prosedur sebagai OBJ_PROCEDURE bertipe TYPE_VOID di scope parent, membuka scope baru, memproses seluruh parameter formal ke dalam btab (dengan flag nrm=0 untuk parameter by-reference), mengeksekusi visitBlock untuk body prosedur, lalu menutup scope dan menghubungkan entri prosedur di tab ke btabIdx yang baru melalui field ref.</td>
</tr>
</table>

```
void SemanticAnalyzer::visitProcDecl(ProcDeclNode* node) {
    auto errCb = [&](const std::string& msg, int line) { this->semanticError(msg, line); };

    // 1. Cek redeklarasi nama prosedur
    if (symTable.lookupCurrentScope(node->name) != -1) {
        semanticError("Identifier '" + node->name + "' already declared in this scope", node->line);
    }

    // 2. Register nama prosedur di scope parent sebelum membuka blok baru
    int procIdx = symTable.enter(node->name, OBJ_PROCEDURE, TYPE_VOID, 0, 1, currentLevel_, 0);

    node->sem.tab_index = procIdx;
    node->sem.type_code = TYPE_VOID;
    node->sem.lev = currentLevel_;

    // 3. Buka scope untuk body prosedur
    int btabIdx = symTable.openBlock();
    currentLevel_++;
    offsetTracker[currentLevel_] = 0;
    int psze = 0;
    int lastParamIdx = -1;

    // 4. Proses parameter prosedur
    for (ParamNode* param : node->params) {
        int param_type_code = TYPE_NONE;

        if (param->typeNode && param->typeNode->kind == ASTKind::SimpleType) {
            auto* st = static_cast<SimpleTypeNode*>(param->typeNode);
            param_type_code = resolveTypeNameHelper(symTable, st->typeName, param->line, errCb);
        }

        int nrm = param->isByRef ? 0 : 1;
        for (const std::string& name : param->names) {
            if (symTable.lookupCurrentScope(name) != -1) {
                semanticError("Identifier '" + name + "' already declared in this scope", param->line);
                continue;
            }
            lastParamIdx = symTable.enter(name, OBJ_VARIABLE, param_type_code, 0, nrm, currentLevel_, psze);
            psze += symTable.typeSize(param_type_code, 0);
        }
    }

    // 5. Simpan metadata parameter ke Btab
    ma3::BtabEntry& bEntry = symTable.getBtabEntry(btabIdx);
    bEntry.psize = psze;
    bEntry.lpar = lastParamIdx;

    // 6. Eksekusi blok prosedur
    if (node->block) {
        visitBlock(node->block);
    }

    // 7. Cleanup scope
    int dummyLastIdx = -1;
    symTable.closeBlock(btabIdx, dummyLastIdx);
    currentLevel_--;
```

```
    // 8. Hubungkan entri prosedur di tabel parent dengan blok baru
    symTable.getTabEntry(procIdx).ref = btabIdx;
    node->sem.ref = btabIdx; // Anotasi ref untuk Decorated AST Printer
}
```

SemanticAnalyzer::visitFuncDecl(FuncDeclNode* node)

```
void SemanticAnalyzer::visitFuncDecl(FuncDeclNode* node) {
    auto errCb = [&](const std::string& msg, int line) { this->semanticError(msg, line); };

    // 1. Cek redeklarasi nama fungsi di scope saat ini
    if (symTable.lookupCurrentScope(node->name) != -1) {
        semanticError("Identifier '" + node->name + "' already declared in this scope", node->line);
    }

    // 2. Evaluasi Return Type
    int retTypeCode = resolveTypeNameHelper(symTable, node->returnTypeName, node->line, errCb);

    // 3. Register nama fungsi di scope parent
    int funcIdx = symTable.enter(node->name, OBJ_FUNCTION, retTypeCode, 0, 1, currentLevel_, 0);

    node->sem.tab_index = funcIdx;
    node->sem.type_code = retTypeCode;
    node->sem.lev = currentLevel_;

    // 4. Buka scope untuk body fungsi
    int btabIdx = symTable.openBlock();
    currentLevel_++;
    offsetTracker[currentLevel_] = 0;
    int psze = 0;
    int lastParamIdx = -1;

    // 5. Proses parameter fungsi
    for (ParamNode* param : node->params) {
        int param_type_code = TYPE_NONE;
        if (param->typeNode && param->typeNode->kind == ASTKind::SimpleType) {
            auto* st = static_cast<SimpleTypeNode*>(param->typeNode);
            param_type_code = resolveTypeNameHelper(symTable, st->typeName, param->line, errCb);
        }

        int nrm = param->isByRef ? 0 : 1;
        for (const std::string& name : param->names) {
            if (symTable.lookupCurrentScope(name) != -1) {
                semanticError("Identifier '" + name + "' already declared in this scope", param->line);
                continue;
            }
            lastParamIdx = symTable.enter(name, OBJ_VARIABLE, param_type_code, 0, nrm, currentLevel_, psze);
            psze += symTable.typeSize(param_type_code, 0);
        }
    }

    // 6. Simpan metadata parameter ke Btab
    ma3::BtabEntry& bEntry = symTable.getBtabEntry(btabIdx);
    bEntry.lpar = lastParamIdx;
    bEntry.psize = psze;

    // 7. Eksekusi blok fungsi
    if (node->block) {
        visitBlock(node->block);
    }
```

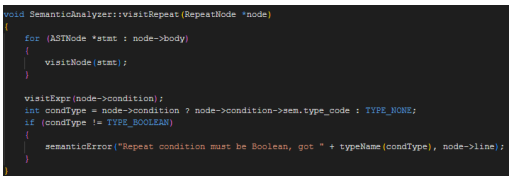
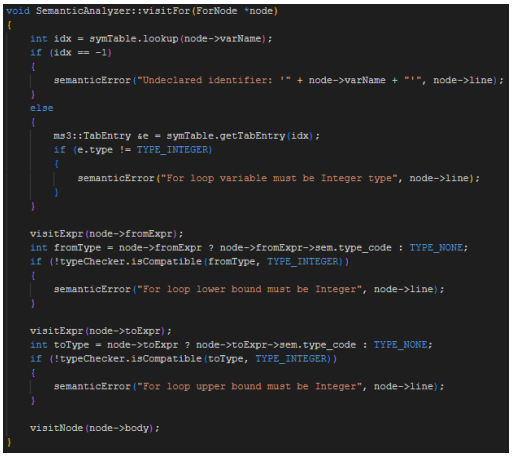
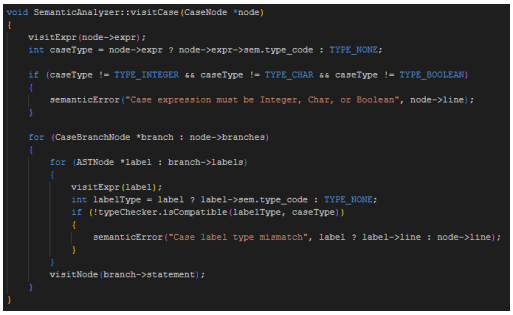
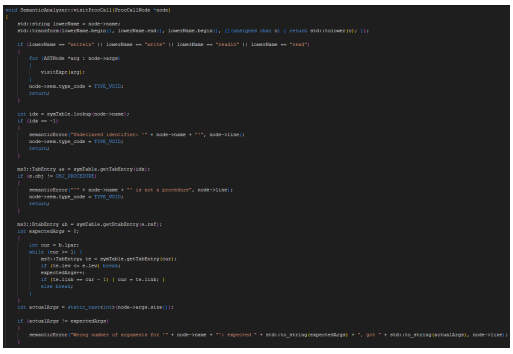
```
    // 8. Cleanup scope
    int dummyLastIdx = -1;
    symTable.closeBlock(btabIdx, dummyLastIdx);
    currentLevel_--;

    // 9. Hubungkan entri fungsi di tabel parent dengan blok baru
    symTable.getTabEntry(funcIdx).ref = btabIdx;
    node->sem.ref = btabIdx;
}
```

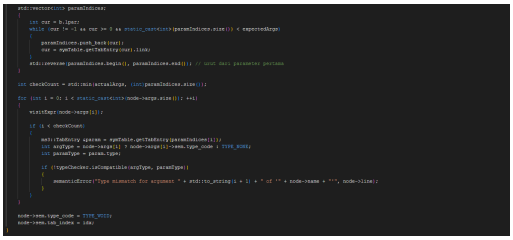
Visitor untuk deklarasi fungsi. Mirip dengan visitProcDecl namun sebelumnya mengevaluasi nama tipe kembalian (returnTypeName) menggunakan resolveTypeNameHelper, lalu mendaftarkan nama fungsi sebagai OBJ_FUNCTION dengan kode tipe hasil tersebut. Blok scope baru dibuka untuk parameter dan body, kemudian ditutup dan dihubungkan ke entri fungsi di scope parent melalui field ref.

2.4.6. File visit_statements.cpp

Tabel 2.4.6. Implementasi kelas visit_statements

Kelas/Fungsi/Prosedur	**Penjelasan**
SemanticAnalyzer::visitCompound(CompoundNode* node) `void SemanticAnalyzer::visitCompound(CompoundNode *node)` `{` `  for (ASTNode *stmt : node->statements)` `  {` `  visitNode(stmt);` `  }` `}`	Visitor untuk blok statement majemuk (begin...end). Mengiterasi seluruh statement di dalam node dan memanggil visitNode untuk masing-masing, sehingga proses analisis semantik berlanjut secara rekursif ke setiap instruksi dalam blok.
SemanticAnalyzer::visitAssign(AssignNode* node) `void SemanticAnalyzer::visitAssign(AssignNode *node)` `{` `  visitExpr(node->target);` `  // Check target is writable (not a constant)` `  if (node->target && node->target->sem.tab_index >= 0)` `  {` `  auto t = symTable.getTabEntry(node->target->sem.tab_index);` `  if (t.obj == OBJ_CONSTANT)` `  {` `  semanticError("Cannot assign to constant '" + t.name + "'", node->line);` `  }` `  }` `  visitExpr(node->value);` `  int targetType = node->target ? node->target->sem.type_node : TYPE_NONE;` `  int valueType = node->value ? node->value->sem.type_node : TYPE_NONE;` `  if (!typeChecker.isAssignmentCompatible(targetType, valueType))` `  {` `  semanticError("Type mismatch in assignment: cannot assign " + typeName(valueType) + " to " + typeName(targetType), node->line);` `  }` `  node->sem.type_node = TYPE_VOID;` `}`	Visitor untuk statement penugasan. Memvalidasi bahwa target tidak berstatus OBJ_CONSTANT, mengevaluasi tipe target dan tipe nilai ekspresi kanan, kemudian memeriksa kompatibilitas tipe menggunakan typeChecker.isAssignmentCompatible(). Kesalahan dilaporkan jika target adalah konstanta atau tipe tidak kompatibel.
SemanticAnalyzer::visitIf(IfNode* node), `void SemanticAnalyzer::visitIf(IfNode *node)` `{` `  visitExpr(node->condition);` `  int condType = node->condition ? node->condition->sem.type_node : TYPE_NONE;` `  if (condType != TYPE_BOOLEAN)` `  {` `  semanticError("If condition must be Boolean, got " + typeName(condType), node->line);` `  }` `  visitNode(node->thenBranch);` `  if (node->elseBranch)` `  {` `  visitNode(node->elseBranch);` `  }` `}` SemanticAnalyzer::visitWhile(WhileNode* node), `void SemanticAnalyzer::visitWhile(WhileNode *node)` `{` `  visitExpr(node->condition);` `  int condType = node->condition ? node->condition->sem.type_node : TYPE_NONE;` `  if (condType != TYPE_BOOLEAN)` `  {` `  semanticError("While condition must be Boolean, got " + typeName(condType), node->line);` `  }` `  visitNode(node->body);` `}` SemanticAnalyzer::visitRepeat(RepeatNode* node)	Visitor untuk struktur percabangan dan pengulangan kondisional. Ketiga fungsi mengevaluasi ekspresi kondisi dan memvalidasi bahwa tipenya adalah TYPE_BOOLEAN. visitIf kemudian mengunjungi thenBranch dan elseBranch (jika ada), visitWhile mengunjungi body loop, sedangkan visitRepeat mengunjungi seluruh pernyataan dalam body sebelum mengevaluasi kondisi terminasi.

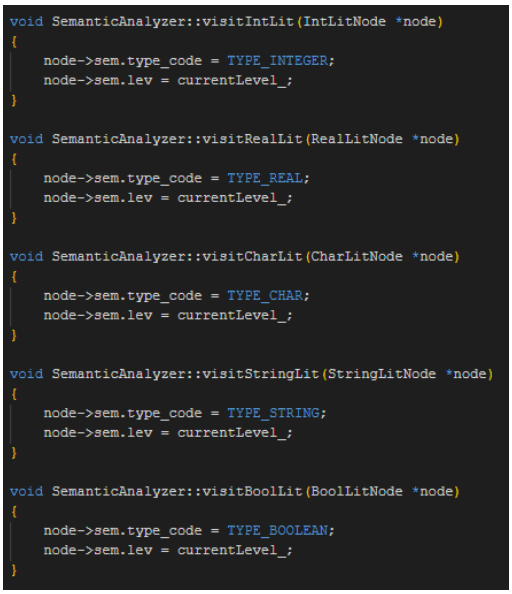
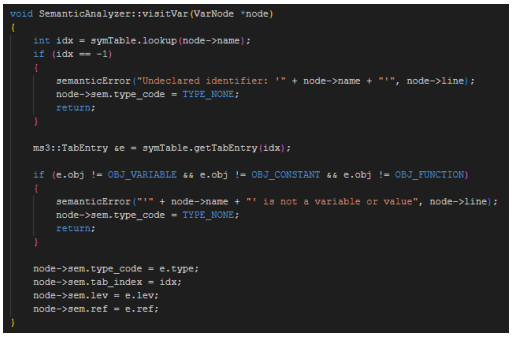
	
SemanticAnalyzer::visitFor(ForNode* node) 	Visitor untuk perulangan for. Memverifikasi bahwa variabel kontrol (varName) telah dideklarasikan dan bertipe Integer. Mengevaluasi ekspresi batas bawah (fromExpr) dan batas atas (toExpr) serta memastikan keduanya kompatibel dengan TYPE_INTEGER, lalu mengunjungi body loop.
SemanticAnalyzer::visitCase(CaseNode* node) 	Visitor untuk statement case. Mengevaluasi tipe ekspresi selektor dan memvalidasi bahwa tipenya adalah INTEGER, CHAR, atau BOOLEAN. Untuk setiap cabang (CaseBranchNode), memeriksa kompatibilitas tipe setiap label dengan tipe selektor, kemudian mengunjungi statement dari masing-masing cabang.
SemanticAnalyzer::visitProcCall(ProcCallNode* node) 	Visitor untuk pemanggilan prosedur. Menangani prosedur built-in (write, writeln, read, readln) secara langsung. Untuk prosedur yang didefinisikan pengguna, memverifikasi bahwa identifier terdaftar sebagai OBJ_PROCEDURE, menghitung jumlah parameter formal dari btab, membandingkannya dengan jumlah argumen aktual, lalu memeriksa kompatibilitas tipe tiap argumen terhadap parameter yang sesuai.

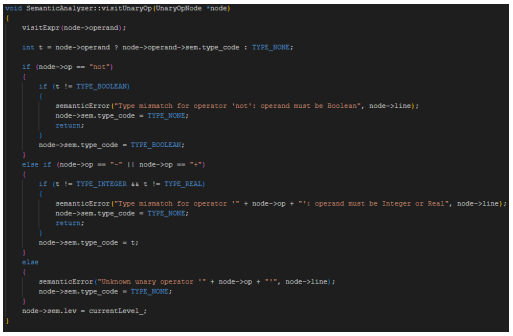
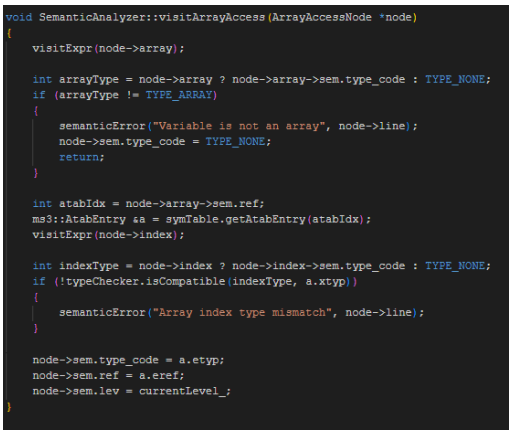
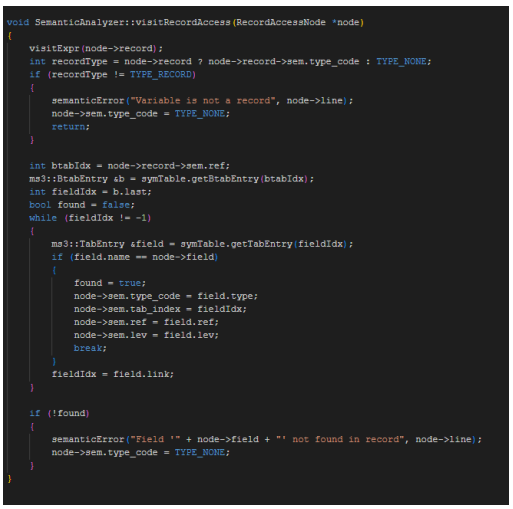
	

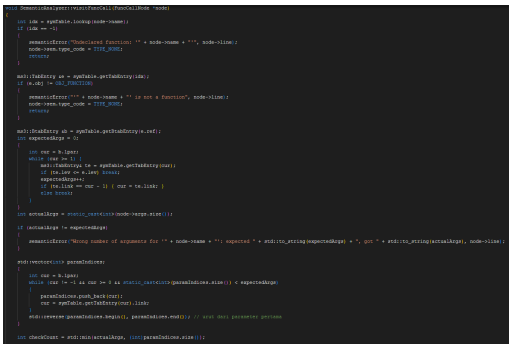
2.4.7. File visit_expressions.cpp

Tabel 2.4.7. Implementasi kelas visit_expressions.cpp

Kelas/Fungsi/Prosedur	**Penjelasan**
SemanticAnalyzer::visitExpr(ASTNode* node) ``` void SemanticAnalyzer::visitExpr(ASTNode *node) { if (!node) return; switch (node->kind) { case ASTKind::BinOp: visitBinOp(static_cast<BinOpNode *>(node)); break; case ASTKind::UnaryOp: visitUnaryOp(static_cast<UnaryOpNode *>(node)); break; case ASTKind::Var: visitVar(static_cast<VarNode *>(node)); break; case ASTKind::ArrayAccess: visitArrayAccess(static_cast<ArrayAccessNode *>(node)); break; case ASTKind::RecordAccess: visitRecordAccess(static_cast<RecordAccessNode *>(node)); break; case ASTKind::FuncCall: visitFuncCall(static_cast<FuncCallNode *>(node)); break; case ASTKind::IntLit: visitIntLit(static_cast<IntLitNode *>(node)); break; case ASTKind::RealLit: visitRealLit(static_cast<RealLitNode *>(node)); break; case ASTKind::CharLit: visitCharLit(static_cast<CharLitNode *>(node)); break; case ASTKind::StringLit: visitStringLit(static_cast<StringLitNode *>(node)); break; case ASTKind::BoolLit: visitBoolLit(static_cast<BoolLitNode *>(node)); break; default: break; } } ```	Fungsi dispatcher utama untuk seluruh ekspresi. Membaca field kind dari node yang diterima dan mendelegasikannya ke visitor ekspresi spesifik yang sesuai menggunakan switch-case, mulai dari BinOp, UnaryOp, Var, ArrayAccess, RecordAccess, FuncCall, hingga seluruh literal primitif (IntLit, RealLit, CharLit, StringLit, BoolLit).
visitIntLit, visitRealLit, visitCharLit, visitStringLit, visitBoolLit	Kumpulan visitor untuk node literal primitif. Masing-masing menetapkan sem.type_code pada node sesuai dengan tipe literalnya: TYPE_INTEGER, TYPE_REAL, TYPE_CHAR, TYPE_STRING, atau TYPE_BOOLEAN. Tidak ada pemeriksaan lanjutan

	yang diperlukan karena tipe literal bersifat statis dan selalu valid.
SemanticAnalyzer::visitVar(VarNode* node) 	Visitor untuk referensi variabel atau identifier. Melakukan lookup di tabel simbol dan memvalidasi bahwa identifier terdaftar serta berkategori OBJ_VARIABLE, OBJ_CONSTANT, atau OBJ_FUNCTION. Mengisi anotasi semantik node (type_code, tab_index, lev, ref) berdasarkan entri yang ditemukan.
SemanticAnalyzer::visitBinOp(BinOpNode* node) 	Visitor untuk ekspresi biner (operator dua operand). Mengevaluasi tipe operand kiri dan kanan, kemudian mendelegasikan pemeriksaan kompatibilitas dan penentuan tipe hasil ke typeChecker.getOperatorResultType(). Jika hasilnya TYPE_NONE, dilaporkan sebagai kesalahan type mismatch.
SemanticAnalyzer::visitUnaryOp(UnaryOpNode* node)	Visitor untuk ekspresi unary (operator satu operand). Memeriksa operator: not hanya valid untuk TYPE_BOOLEAN, sedangkan - dan + hanya valid untuk TYPE_INTEGER atau TYPE_REAL. Tipe hasil ditetapkan sama dengan tipe operand untuk

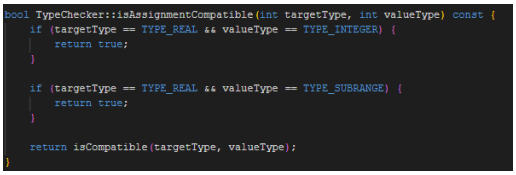
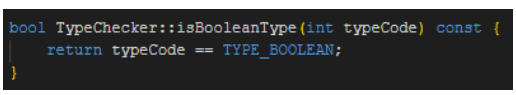
	operator numerik, atau TYPE_BOOLEAN untuk not.
SemanticAnalyzer::visitArrayAccess(ArrayAccessNode* node) 	Visitor untuk akses elemen array (a[i]). Memverifikasi bahwa ekspresi array bertipe TYPE_ARRAY, mengambil informasi AtabEntry dari atab berdasarkan ref, lalu memeriksa kompatibilitas tipe ekspresi indeks dengan xtyp (tipe indeks array). Tipe hasil ditetapkan sebagai etyp (tipe elemen array).
SemanticAnalyzer::visitRecordAccess(RecordAccessNode* node) 	Visitor untuk akses field record (r.field). Memverifikasi bahwa ekspresi record bertipe TYPE_RECORD, lalu menelusuri linked list field di dalam btab yang ditunjuk oleh ref untuk menemukan field dengan nama yang sesuai. Jika field tidak ditemukan, dilaporkan sebagai kesalahan; jika ditemukan, tipe hasil ditetapkan dari field tersebut.
SemanticAnalyzer::visitFuncCall(FuncCallNode* node)	Visitor untuk pemanggilan fungsi sebagai ekspresi. Memverifikasi bahwa identifier terdaftar sebagai

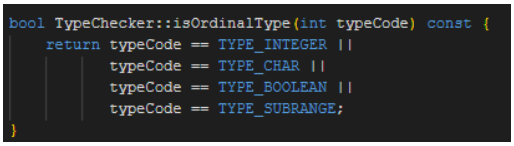
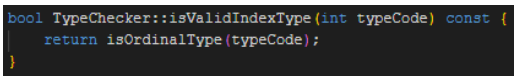
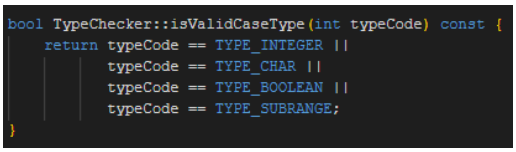
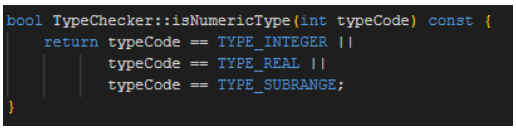
 	OBJ_FUNCTION, menghitung jumlah parameter formal dari btab, membandingkan dengan jumlah argumen aktual, dan memeriksa kompatibilitas tipe tiap argumen. Tipe hasil ekspresi ditetapkan sebagai tipe kembalian fungsi (e.type).

2.4.8. File TypeChecker.hpp dan TypeChecker.cpp

Tabel 2.4.8. Implementasi kelas TypeChecker

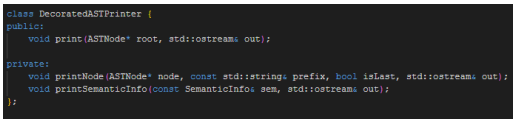
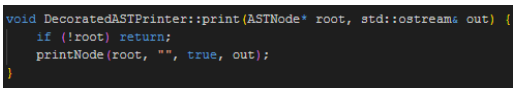
Kelas/Fungsi/Prosedur	**Penjelasan**
class TypeChecker ``` class TypeChecker { public: TypeChecker() = default; // cek compatible dari dua tipe (untuk perbandingan, case label, dll) bool isCompatible(int typeCode1, int typeCode2) const; // cek valueType bisa di-assign ke targetType atau tidak bool isAssignmentCompatible(int targetType, int valueType) const; // ambil tipe hasil dari operator binary dengan operand kiri dan kanan int getOperatorResultType(const std::string& op, int leftType, int rightType) const; // cek tipe valid sebagai kondisi boolean (if, while, repeat) bool isBooleanType(int typeCode) const; // cek tipe termasuk ordinal atau tidak bool isOrdinalType(int typeCode) const; // cek tipe valid sebagai index array atau tidak bool isValidIndexType(int typeCode) const; // cek tipe valid sebagai ekspresi case atau tidak bool isValidCaseType(int typeCode) const; // cek tipe termasuk numerik atau tidak bool isNumericType(int typeCode) const; }; ```	Kelas utilitas yang bertanggung jawab untuk memusatkan aturan kompatibilitas tipe pada proses analisis semantik. Kelas ini digunakan untuk memeriksa assignment, ekspresi, operator, kondisi boolean, tipe ordinal, indeks array, dan ekspresi case agar keputusan type checking tidak tersebar di banyak visitor.
bool TypeChecker::isCompatible(int typeCode1, int typeCode2) const ``` bool TypeChecker::isCompatible(int typeCode1, int typeCode2) const { if (typeCode1 == typeCode2) { return true; } if (isIntegerBased(typeCode1) && isIntegerBased(typeCode2)) { return true; } return false; } ```	Mengecek apakah dua tipe dapat dianggap kompatibel. Dua tipe dinyatakan kompatibel apabila memiliki kode tipe yang sama, atau keduanya termasuk tipe berbasis integer, yaitu Integer dan Subrange. Fungsi ini digunakan untuk pemeriksaan relasi umum seperti perbandingan, label case, dan kecocokan tipe dasar.
bool	Mengecek apakah suatu nilai boleh diberikan ke tipe

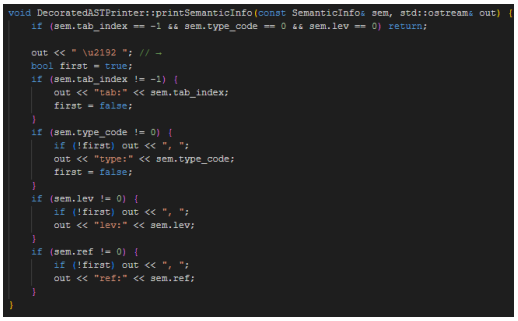
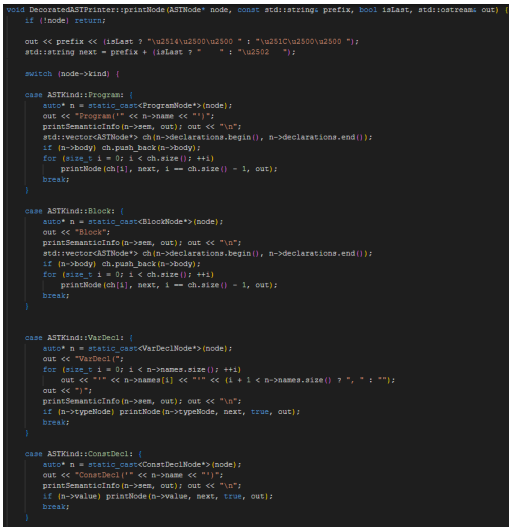
TypeChecker::isAssignmentCompatible(int targetType, int valueType) const 	target pada statement assignment. Selain kompatibilitas umum, fungsi ini juga mengizinkan pelebaran tipe dari Integer atau Subrange ke Real.
int TypeChecker::getOperatorResultType(const std::string& op, int leftType, int rightType) const 	Menentukan tipe hasil dari operasi binary berdasarkan operator dan tipe operand kiri-kanan. Operator aritmetika menghasilkan Integer atau Real sesuai operand, operator pembagian menghasilkan Real, operator div dan mod hanya menerima tipe integer-based, operator logika menghasilkan Boolean, dan operator relasional menghasilkan Boolean jika operand valid. Jika kombinasi operand tidak sah, fungsi mengembalikan TYPE_NONE.
bool TypeChecker::isBooleanType(int typeCode) const 	Memvalidasi apakah suatu tipe merupakan Boolean. Fungsi ini dipakai untuk kebutuhan ekspresi kondisi seperti if, while, dan repeat-until.
bool TypeChecker::isOrdinalType(int typeCode) const	Memeriksa apakah suatu tipe termasuk ordinal, yaitu Integer, Char, Boolean, atau Subrange.

	
bool TypeChecker::isValidIndexType(int typeCode) const 	Memvalidasi tipe yang dapat digunakan sebagai indeks array. Implementasi menggunakan aturan ordinal, sehingga tipe indeks array harus berupa Integer, Char, Boolean, atau Subrange.
bool TypeChecker::isValidCaseType(int typeCode) const 	Memvalidasi tipe ekspresi case. Tipe yang diterima adalah Integer, Char, Boolean, atau Subrange.
bool TypeChecker::isNumericType(int typeCode) const 	Memeriksa apakah suatu tipe termasuk numerik, yaitu Integer, Real, atau Subrange. Fungsi ini menjadi dasar pemeriksaan operator aritmetika dan relasional numerik.

2.4.9. File DecoratedASTPrinter.hpp dan DecoratedASTPrinter.cpp

Tabel 2.4.9. Implementasi kelas DecoratedASTPrinter

Kelas/Fungsi/Prosedur	**Penjelasan**
class DecoratedASTPrinter 	Kelas utilitas publik yang bertanggung jawab atas penulisan luaran (output) AST yang sudah teranotasi ke standar aliran output (stdout) dalam bentuk visual pohon.
void DecoratedASTPrinter::print(ASTNode* root, std::ostream& out) 	Fungsi penggerak logika percetakan yang merangkai indentasi setiap tingkat AST serta mengatur garis percabangan node secara rekursif.
void	Fungsi cetak privat yang secara spesifik menampilkan

DecoratedASTPrinter::printSemanticInfo(const SemanticInfo& sem, std::ostream& out) 	nilai anotasi semantik (tipe, indeks tabel simbol, level leksikal) di sebelah simpul node saat dicetak. Format yang dihasilkan mirip dengan fitur cetak TreePrinter pada Milestone 2, tetapi method ini melampirkan informasi struktur semantiknya.
void DecoratedASTPrinter::printNode(ASTNode* node, const std::string& prefix, bool isLast, std::ostream& out) 	Fungsi privat rekursif yang merupakan inti mekanisme pencetakan pohon AST berdekorasi. Menggunakan karakter Unicode box-drawing untuk membangun visualisasi hierarki pohon secara indentasi bertingkat. Untuk setiap node, fungsi membaca kind-nya, mencetak label yang sesuai beserta anotasi semantik via printSemanticInfo(), lalu memanggil dirinya sendiri secara rekursif untuk seluruh anak node dengan prefix yang sudah diperpanjang.

```
    case ASTKind::TypeDecl: {
        auto* n = static_cast<TypeDeclNode*>(node);
        out << "TypeDecl('" << n->name << "')";
        printSemanticInfo(n->sem, out); out << "\n";
        if (n->typeNode) printNode(n->typeNode, next, true, out);
        break;
    }

    case ASTKind::ProcDecl: {
        auto* n = static_cast<ProcDeclNode*>(node);
        out << "ProcDecl('" << n->name << "')";
        printSemanticInfo(n->sem, out); out << "\n";
        size_t total = n->params.size() + (n->block ? 1 : 0);
        size_t i = 0;
        for (ParamNode* p : n->params) {
            printNode(p, next, ++i == total, out);
        }
        if (n->block) printNode(n->block, next, true, out);
        break;
    }

    case ASTKind::FuncDecl: {
        auto* n = static_cast<FuncDeclNode*>(node);
        out << "FuncDecl('" << n->name << "', returns: " << n->returnTypeName << ")";
        printSemanticInfo(n->sem, out); out << "\n";
        size_t total = n->params.size() + (n->block ? 1 : 0);
        size_t i = 0;
        for (ParamNode* p : n->params) {
            printNode(p, next, ++i == total, out);
        }
        if (n->block) printNode(n->block, next, true, out);
        break;
    }

    case ASTKind::Param: {
        auto* n = static_cast<ParamNode*>(node);
        out << "Param(" << (n->isByRef ? "var " : "");
        for (size_t i = 0; i < n->names.size(); ++i)
            out << "'" << n->names[i] << "'" << (i + 1 < n->names.size() ? ", " : "");
        out << ")";
        printSemanticInfo(n->sem, out); out << "\n";
        if (n->typeNode) printNode(n->typeNode, next, true, out);
        break;
    }

    case ASTKind::SimpleType: {
        auto* n = static_cast<SimpleTypeNode*>(node);
        out << "SimpleType('" << n->typeName << "')";
        printSemanticInfo(n->sem, out); out << "\n";
        break;
    }
```

```
    case ASTKind::ArrayType: {
        auto* n = static_cast<ArrayTypeNode*>(node);
        out << "ArrayType";
        printSemanticInfo(n->sem, out); out << "\n";
        if (n->indexType)   printNode(n->indexType,   next, !n->elementType, out);
        if (n->elementType) printNode(n->elementType, next, true, out);
        break;
    }

    case ASTKind::RecordType: {
        auto* n = static_cast<RecordTypeNode*>(node);
        out << "RecordType";
        printSemanticInfo(n->sem, out); out << "\n";
        for (size_t i = 0; i < n->fields.size(); ++i)
            printNode(n->fields[i], next, i == n->fields.size() - 1, out);
        break;
    }

    case ASTKind::Range: {
        auto* n = static_cast<RangeNode*>(node);
        out << "Range";
        printSemanticInfo(n->sem, out); out << "\n";
        if (n->low)  printNode(n->low,  next, !n->high, out);
        if (n->high) printNode(n->high, next, true, out);
        break;
    }

    case ASTKind::Enumerated: {
        auto* n = static_cast<EnumeratedNode*>(node);
        out << "Enumerated(";
        for (size_t i = 0; i < n->values.size(); ++i)
            out << "'" << n->values[i] << "'" << (i + 1 < n->values.size() ? ", " : "");
        out << ")";
        printSemanticInfo(n->sem, out); out << "\n";
        // No AST children — values are plain strings
        break;
    }

    case ASTKind::FieldDecl: {
        auto* n = static_cast<FieldDeclNode*>(node);
        out << "FieldDecl(";
        for (size_t i = 0; i < n->names.size(); ++i)
            out << "'" << n->names[i] << "'" << (i + 1 < n->names.size() ? ", " : "");
        out << ")";
        printSemanticInfo(n->sem, out); out << "\n";
        if (n->typeNode) printNode(n->typeNode, next, true, out);
        break;
    }
```

```
    case ASTKind::Compound: {
        auto* n = static_cast<CompoundNode*>(node);
        out << "Compound";
        printSemanticInfo(n->sem, out); out << "\n";
        for (size_t i = 0; i < n->statements.size(); ++i)
            printNode(n->statements[i], next, i == n->statements.size() - 1, out);
        break;
    }

    case ASTKind::Assign: {
        auto* n = static_cast<AssignNode*>(node);
        out << "Assign";
        printSemanticInfo(n->sem, out); out << "\n";
        if (n->target) printNode(n->target, next, !n->value, out);
        if (n->value)  printNode(n->value,  next, true, out);
        break;
    }

    case ASTKind::If: {
        auto* n = static_cast<IfNode*>(node);
        out << "If";
        printSemanticInfo(n->sem, out); out << "\n";
        bool hasElse = n->elseBranch != nullptr;
        if (n->condition)  printNode(n->condition,  next, !n->thenBranch && !hasElse, out);
        if (n->thenBranch) printNode(n->thenBranch, next, !hasElse, out);
        if (n->elseBranch) printNode(n->elseBranch, next, true, out);
        break;
    }

    case ASTKind::While: {
        auto* n = static_cast<WhileNode*>(node);
        out << "While";
        printSemanticInfo(n->sem, out); out << "\n";
        if (n->condition) printNode(n->condition, next, !n->body, out);
        if (n->body)      printNode(n->body,      next, true, out);
        break;
    }

    case ASTKind::For: {
        auto* n = static_cast<ForNode*>(node);
        out << "For('" << n->varName << "', " << (n->downto ? "downto" : "to") << ")";
        printSemanticInfo(n->sem, out); out << "\n";
        if (n->fromExpr) printNode(n->fromExpr, next, !n->toExpr && !n->body, out);
        if (n->toExpr)   printNode(n->toExpr,   next, !n->body, out);
        if (n->body)     printNode(n->body,     next, true, out);
        break;
    }
```

```
    case ASTKind::Repeat: {
        auto* n = static_cast<RepeatNode*>(node);
        out << "Repeat";
        printSemanticInfo(n->sem, out); out << "\n";
        for (size_t i = 0; i < n->body.size(); ++i)
            printNode(n->body[i], next, !n->condition && i == n->body.size() - 1, out);
        if (n->condition) printNode(n->condition, next, true, out);
        break;
    }

    case ASTKind::Case: {
        auto* n = static_cast<CaseNode*>(node);
        out << "Case";
        printSemanticInfo(n->sem, out); out << "\n";
        if (n->expr) printNode(n->expr, next, n->branches.empty(), out);
        for (size_t i = 0; i < n->branches.size(); ++i)
            printNode(n->branches[i], next, i == n->branches.size() - 1, out);
        break;
    }

    case ASTKind::CaseBranch: {
        auto* n = static_cast<CaseBranchNode*>(node);
        out << "CaseBranch";
        printSemanticInfo(n->sem, out); out << "\n";
        for (size_t i = 0; i < n->labels.size(); ++i)
            printNode(n->labels[i], next, !n->statement && i == n->labels.size() - 1, out);
        if (n->statement) printNode(n->statement, next, true, out);
        break;
    }

    case ASTKind::ProcCall: {
        auto* n = static_cast<ProcCallNode*>(node);
        out << "ProcCall(" << n->name << ")";
        printSemanticInfo(n->sem, out); out << "\n";
        for (size_t i = 0; i < n->args.size(); ++i)
            printNode(n->args[i], next, i == n->args.size() - 1, out);
        break;
    }

    case ASTKind::BinOp: {
        auto* n = static_cast<BinOpNode*>(node);
        out << "BinOp(" << n->op << ")";
        printSemanticInfo(n->sem, out); out << "\n";
        if (n->left)  printNode(n->left,  next, !n->right, out);
        if (n->right) printNode(n->right, next, true, out);
        break;
    }
```

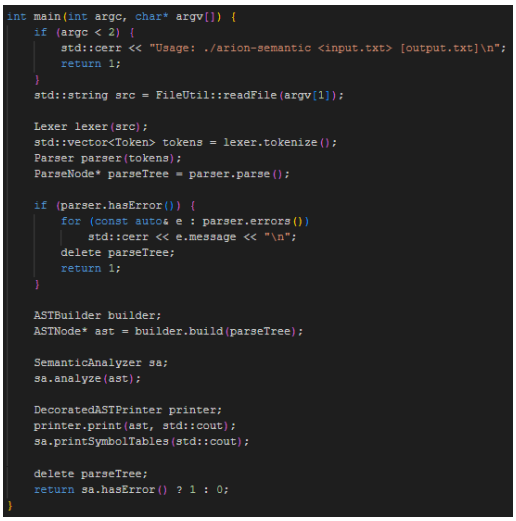
```
    case ASTKind::UnaryOp: {
        auto* n = static_cast<UnaryOpNode*>(node);
        out << "UnaryOp(" << n->op << ")";
        printSemanticInfo(n->sem, out); out << "\n";
        if (n->operand) printNode(n->operand, next, true, out);
        break;
    }

    case ASTKind::Var: {
        auto* n = static_cast<VarNode*>(node);
        out << "Var(" << n->name << ")";
        printSemanticInfo(n->sem, out); out << "\n";
        break;
    }

    case ASTKind::ArrayAccess: {
        auto* n = static_cast<ArrayAccessNode*>(node);
        out << "ArrayAccess";
        printSemanticInfo(n->sem, out); out << "\n";
        if (n->array) printNode(n->array, next, !n->index, out);
        if (n->index) printNode(n->index, next, true, out);
        break;
    }

    case ASTKind::RecordAccess: {
        auto* n = static_cast<RecordAccessNode*>(node);
        out << "RecordAccess(" << n->field << ")";
        printSemanticInfo(n->sem, out); out << "\n";
        if (n->record) printNode(n->record, next, true, out);
        break;
    }

    case ASTKind::IntLit: {
        auto* n = static_cast<IntLitNode*>(node);
        out << "IntLit(" << n->value << ")";
        printSemanticInfo(n->sem, out); out << "\n";
        break;
    }

    case ASTKind::RealLit: {
        auto* n = static_cast<RealLitNode*>(node);
        out << "RealLit(" << n->value << ")";
        printSemanticInfo(n->sem, out); out << "\n";
        break;
    }

    case ASTKind::CharLit: {
        auto* n = static_cast<CharLitNode*>(node);
        out << "CharLit(" << n->value << ")";
        printSemanticInfo(n->sem, out); out << "\n";
        break;
    }
```

```
    case ASTKind::StringLit: {
        auto* n = static_cast<StringLitNode*>(node);
        out << "StringLit(" << n->value << ")";
        printSemanticInfo(n->sem, out); out << "\n";
        break;
    }

    case ASTKind::BoolLit: {
        auto* n = static_cast<BoolLitNode*>(node);
        out << "BoolLit(" << (n->value ? "true" : "false") << ")";
        printSemanticInfo(n->sem, out); out << "\n";
        break;
    }

    case ASTKind::FuncCall: {
        auto* n = static_cast<FuncCallNode*>(node);
        out << "FuncCall(" << n->name << ")";
        printSemanticInfo(n->sem, out); out << "\n";
        for (size_t i = 0; i < n->args.size(); ++i)
            printNode(n->args[i], next, i == n->args.size() - 1, out);
        break;
    }

    } // end switch
}
```

2.4.10. File main.cpp

Tabel 2.4.10. Implementasi kelas main

Kelas/Fungsi/Prosedur	**Penjelasan**

int main(int argc, char* argv[]) 	Alur *executable* kompilasi Arion Semantic yang mengatur jalannya program utama sebagai proses sekuensial yang merangkai pipeline MS1, MS2, hingga MS3 secara berurutan. Program membaca berkas *source code*, melaksanakan *tokenizer* dari Lexer, menghasilkan *parse tree* dari Parser, kemudian merakit *AST tree* di dalam *ASTBuilder*, dan menempelkan metadata melalui *SemanticAnalyzer*. Di akhir, mencetak visualisasi decorated tree dan hasil dari tabel simbol.

```
int main(int argc, char* argv[]) {
    if (argc < 2) {
        std::cerr << "Usage: ./arion-semantic <input.txt> [output.txt]\n";
        return 1;
    }
    std::string src = FileUtil::readFile(argv[1]);

    Lexer lexer(src);
    std::vector<Token> tokens = lexer.tokenize();
    Parser parser(tokens);
    ParseNode* parseTree = parser.parse();

    if (parser.hasError()) {
        for (const auto& e : parser.errors())
            std::cerr << e.message << "\n";
        delete parseTree;
        return 1;
    }

    ASTBuilder builder;
    ASTNode* ast = builder.build(parseTree);

    SemanticAnalyzer sa;
    sa.analyze(ast);

    DecoratedASTPrinter printer;
    printer.print(ast, std::cout);
    sa.printSymbolTables(std::cout);

    delete parseTree;
    return sa.hasError() ? 1 : 0;
}
```

2.4.Alur Kerja Program

Alur kerja program semantic analyzer adalah sebagai berikut:

1. Program menerima path file source code dalam bahasa pemrograman Arion sebagai command line argument pertama.
2. Fungsi FileUtil::readFile() membaca seluruh konten file ke dalam string. Jika file tidak ditemukan atau tidak dapat dibaca, program akan melempar exception untuk menghasilkan pesan error dan menghentikan program dengan exit code 1.
3. String source code diproses oleh Lexer yang melakukan scanning karakter per karakter untuk mengidentifikasi pola yang sesuai dengan definisi token bahasa Arion, lalu menghasilkan std::vector<Token>.
4. Vector token diteruskan ke Parser yang memproses token secara recursive descent dan membangun parse tree sesuai grammar bahasa Arion.
5. Parse tree diserahkan ke ASTBuilder yang melakukan traversal depth-first untuk mengonversi setiap node parse tree menjadi node AST konkret sesuai jenisnya, menghasilkan sebuah AST yang lebih ringkas.
6. SymbolTable diinisialisasi dengan tipe dan konstanta bawaan (Integer, Real, Char, Boolean, String, True, False), kemudian blok global dibuka sebagai scope awal melalui openBlock().

7. SemanticAnalyzer memulai traversal AST dari visitProgram() secara rekursif ke seluruh deklarasi dan pernyataan dalam program.

8. Setiap deklarasi variabel, konstanta, tipe, prosedur, dan fungsi yang ditemukan didaftarkan ke SymbolTable melalui enter(). Untuk prosedur dan fungsi, blok baru dibuka dengan openBlock() saat masuk ke scope-nya dan ditutup kembali dengan closeBlock() setelah selesai, disertai kenaikan dan penurunan level leksikal.

9. Setiap ekspresi dan pernyataan divalidasi kesesuaian tipenya oleh TypeChecker, mencakup operasi binary, penugasan, kondisi boolean, indeks array, dan akses record field.

10. Hasil anotasi semantik berupa kode tipe, indeks tabel simbol, level leksikal, dan referensi tabel pendukung dituliskan ke field sem pada masing-masing node AST.

11. Seluruh error semantik yang ditemukan dikumpulkan ke dalam sebuah std::vector<std::string> melalui semanticError() tanpa menghentikan traversal, sehingga semua kesalahan dapat dilaporkan sekaligus di akhir eksekusi.

12. Setelah traversal selesai, program memeriksa keberadaan error melalui hasError(). Jika terdapat error, seluruh pesan error dicetak ke standard error dan program dihentikan dengan exit code 1. Jika tidak ada error, DecoratedASTPrinter mencetak AST yang telah dianotasi beserta dump ketiga tabel simbol (tab, btab, atab) sebagai keluaran diagnostik, dan program diakhiri dengan exit code 0.

2.5.Justifikasi Implementasi

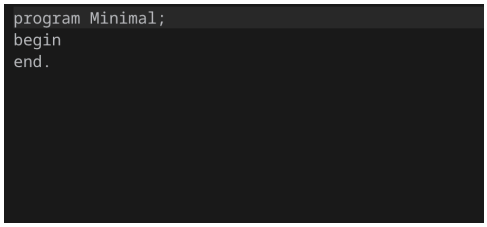
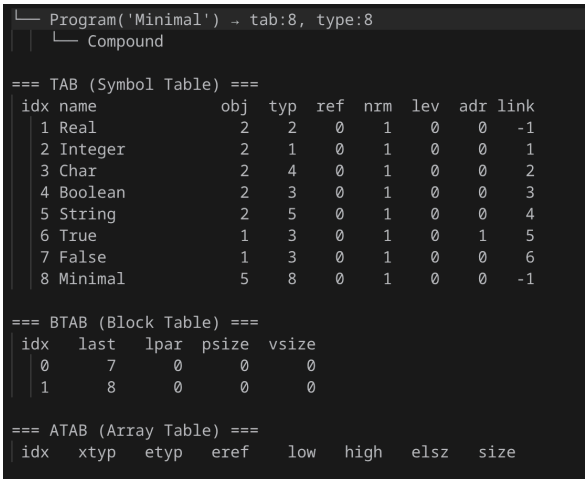
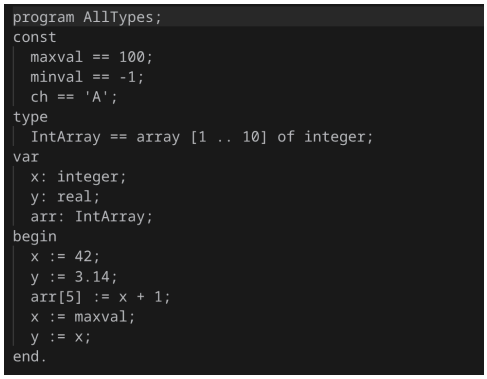
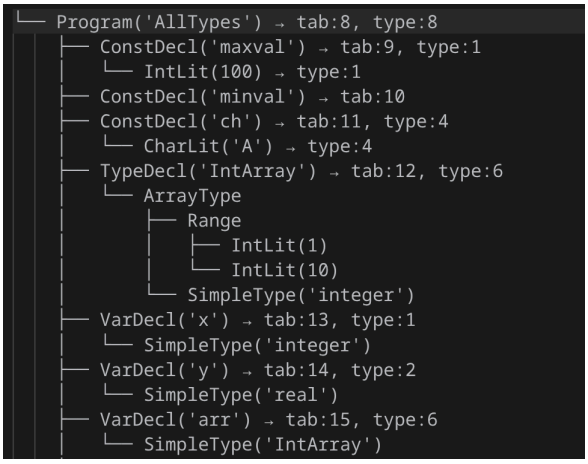
Program semantic analyzer ditulis dengan bahasa pemrograman C++ mengikuti milestone-milestone sebelumnya. Pendekatan visitor secara rekursif dipilih agar alur pengecekan (deklarasi → blok → statement → ekspresi) mengikuti struktur AST, dan memungkinkan push/pop scope yang eksplisit untuk menangani variabel lokal, parameter, array, dan record. Dekorasi AST dengan metadata (type_code, referensi simbol, level scope) memudahkan visualisasi dan menjadi penghubung ke fase berikutnya. Desain tabel simbol mengadopsi tiga tabel terpisah, yaitu tab, btab, dan atab, yang terinspirasi dari pendekatan Wirth. Pemisahan ini memungkinkan representasi cakupan leksikal, metadata blok, dan informasi *array* secara mandiri tanpa redundansi data. Pengelolaan *scope* dilakukan dengan mekanisme *stack* internal melalui openBlock() dan closeBlock() karena struktur *stack* secara alami merepresentasikan sifat leksikal bersarang dari bahasa Arion. Pemeriksaan tipe dipisahkan ke dalam kelas TypeChecker tersendiri

karena logika kompatibilitas tipe bersifat ortogonal terhadap traversal AST, sehingga aturan tipe dapat dimodifikasi atau diperluas tanpa mengubah alur analisis utama. Kumpulan *error* semantik (*undeclared identifier*, *type mismatch*, operasi tidak valid, dll.) diakumulasikan ke dalam sebuah vector tanpa menghentikan traversal, agar *error handling* terstandarisasi dan pesan yang dihasilkan lebih informatif untuk pengguna.

BAB 3

PENGUJIAN

Tabel 3.1. Pengujian program

No	Input	Output
1		
2		

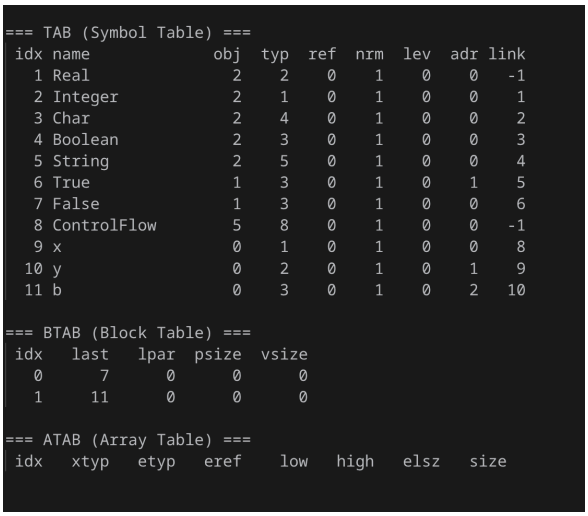
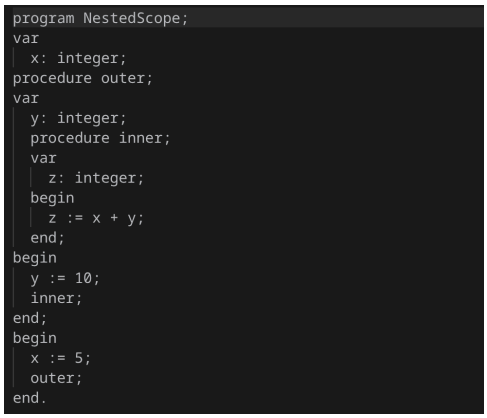
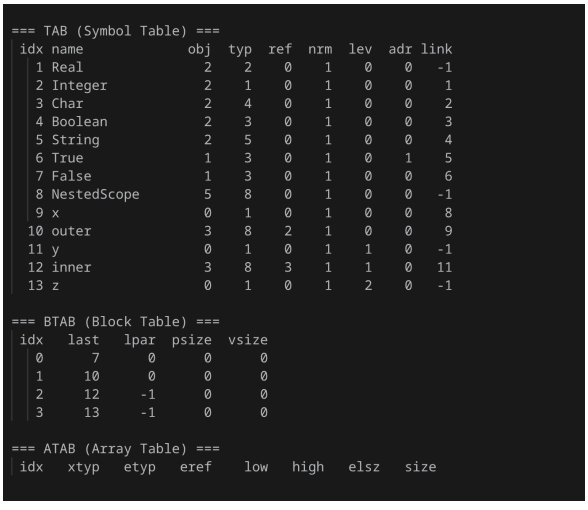
		└── Compound ├── Assign → type:8 │ ├── Var('x') → tab:13, type:1 │ └── IntLit(42) → type:1 ├── Assign → type:8 │ ├── Var('y') → tab:14, type:2 │ └── RealLit(3.14) → type:2 ├── Assign → type:8 │ ├── ArrayAccess → type:1 │ │ ├── Var('arr') → tab:15, type:6 │ │ └── IntLit(5) → type:1 │ └── BinOp('+') → type:1 │ ├── Var('x') → tab:13, type:1 │ └── IntLit(1) → type:1 ├── Assign → type:8 │ ├── Var('x') → tab:13, type:1 │ └── Var('maxval') → tab:9, type:1 └── Assign → type:8 ├── Var('y') → tab:14, type:2 └── Var('x') → tab:13, type:1 === TAB (Symbol Table) === idx name obj typ ref nrm lev adr link 1 Real 2 2 0 1 0 0 -1 2 Integer 2 1 0 1 0 0 1 3 Char 2 4 0 1 0 0 2 4 Boolean 2 3 0 1 0 0 3 5 String 2 5 0 1 0 0 4 6 True 1 3 0 1 0 1 5 7 False 1 3 0 1 0 0 6 8 AllTypes 5 8 0 1 0 0 -1 9 maxval 1 1 0 1 0 100 8 10 minval 1 0 0 1 0 0 9 11 ch 1 4 0 1 0 65 10 12 IntArray 2 6 0 1 0 0 11 13 x 0 1 0 1 0 0 12 14 y 0 2 0 1 0 1 13 15 arr 0 6 0 1 0 2 14 === BTAB (Block Table) === idx last lpar psize vsize 0 7 0 0 0 1 15 0 0 0 === ATAB (Array Table) === idx xtyp etyp eref low high elsz size 0 1 1 0 1 10 1 10
3	program ProcAndFunc; var result: integer; procedure greet; begin end; procedure setVal(n: integer); var t: integer; begin t := n; end; function add(a: integer; b: integer): integer; begin end; begin greet; setVal(5); result := add(3, 4); end.	└── Program('ProcAndFunc') → tab:8, type:8 ├── VarDecl('result') → tab:9, type:1 │ └── SimpleType('integer') ├── ProcDecl('greet') → tab:10, type:8, ref:2 │ └── Block │ └── Compound ├── ProcDecl('setVal') → tab:11, type:8, ref:3 │ ├── Param('n') │ │ └── SimpleType('integer') │ └── Block │ ├── VarDecl('t') → tab:13, type:1, lev:1 │ │ └── SimpleType('integer') │ └── Compound │ └── Assign → type:8 │ ├── Var('t') → tab:13, type:1, lev:1 │ └── Var('n') → tab:12, type:1, lev:1 ├── FuncDecl('add', returns: integer) → tab:14, type:1, ref:4 │ ├── Param('a') │ │ └── SimpleType('integer') │ ├── Param('b') │ │ └── SimpleType('integer') │ └── Block │ └── Compound └── Compound

		└── Compound ├── ProcCall('greet') → tab:10, type:8 ├── ProcCall('setVal') → tab:11, type:8 │ └── IntLit(5) → type:1 └── Assign → type:8 ├── Var('result') → tab:9, type:1 └── FuncCall('add') → tab:14, type:1 ├── IntLit(3) → type:1 └── IntLit(4) → type:1 === TAB (Symbol Table) === idx name obj typ ref nrm lev adr link 1 Real 2 2 0 1 0 0 -1 2 Integer 2 1 0 1 0 0 1 3 Char 2 4 0 1 0 0 2 4 Boolean 2 3 0 1 0 0 3 5 String 2 5 0 1 0 0 4 6 True 1 3 0 1 0 1 5 7 False 1 3 0 1 0 0 6 8 ProcAndFunc 5 8 0 1 0 0 -1 9 result 0 1 0 1 0 0 8 10 greet 3 8 2 1 0 0 9 11 setVal 3 8 3 1 0 0 10 12 n 0 1 0 1 1 0 -1 13 t 0 1 0 1 1 0 12 14 add 4 1 4 1 0 0 11 15 a 0 1 0 1 1 0 -1 16 b 0 1 0 1 1 1 15 === BTAB (Block Table) === idx last lpar psize vsize 0 7 0 0 0 1 14 0 0 0 2 -1 -1 0 0 3 13 12 1 0 4 16 16 2 0 === ATAB (Array Table) === idx xtyp etyp eref low high elsz size
4	program ControlFlow; var x: integer; y: real; b: boolean; begin x := 5; if x == 5 then x := 10; if x > 0 then y := 3.14 else y := 0.0; while x > 0 do x := x - 1; for x := 1 to 10 do y := y + 1.0; for x := 10 downto 1 do y := y - 1.0; repeat x := x + 1 until x == 20; case x of 1: y := 1.0; 2: y := 2.0 end; b := x > 0; b := not b; b := (x == 1) and (y == 2.0); b := (x == 1) or (y == 2.0); end.	└── Program('ControlFlow') → tab:8, type:8 ├── VarDecl('x') → tab:9, type:1 │ └── SimpleType('integer') ├── VarDecl('y') → tab:10, type:2 │ └── SimpleType('real') ├── VarDecl('b') → tab:11, type:3 │ └── SimpleType('boolean') └── Compound ├── Assign → type:8 │ ├── Var('x') → tab:9, type:1 │ └── IntLit(5) → type:1 ├── If │ ├── BinOp('==') → type:3 │ │ ├── Var('x') → tab:9, type:1 │ │ └── IntLit(5) → type:1 │ └── Assign → type:8 │ ├── Var('x') → tab:9, type:1 │ └── IntLit(10) → type:1 ├── If │ ├── BinOp('>') → type:3 │ │ ├── Var('x') → tab:9, type:1 │ │ └── IntLit(0) → type:1 │ ├── Assign → type:8 │ │ ├── Var('y') → tab:10, type:2 │ │ └── RealLit(3.14) → type:2 │ └── Assign → type:8 │ ├── Var('y') → tab:10, type:2 │ └── RealLit(0) → type:2

```
    ├── While
    │   ├── BinOp('>') → type:3
    │   │   ├── Var('x') → tab:9, type:1
    │   │   └── IntLit(0) → type:1
    │   └── Assign → type:8
    │       ├── Var('x') → tab:9, type:1
    │       └── BinOp('-') → type:1
    │           ├── Var('x') → tab:9, type:1
    │           └── IntLit(1) → type:1
    ├── For('x', to)
    │   ├── IntLit(1) → type:1
    │   ├── IntLit(10) → type:1
    │   └── Assign → type:8
    │       ├── Var('y') → tab:10, type:2
    │       └── BinOp('+') → type:2
    │           ├── Var('y') → tab:10, type:2
    │           └── RealLit(1) → type:2
    ├── For('x', downto)
    │   ├── IntLit(10) → type:1
    │   ├── IntLit(1) → type:1
    │   └── Assign → type:8
    │       ├── Var('y') → tab:10, type:2
    │       └── BinOp('-') → type:2
    │           ├── Var('y') → tab:10, type:2
    │           └── RealLit(1) → type:2
```

```
    ├── Repeat
    │   ├── Assign → type:8
    │   │   ├── Var('x') → tab:9, type:1
    │   │   └── BinOp('+') → type:1
    │   │       ├── Var('x') → tab:9, type:1
    │   │       └── IntLit(1) → type:1
    │   └── BinOp('==') → type:3
    │       ├── Var('x') → tab:9, type:1
    │       └── IntLit(20) → type:1
    ├── Case
    │   ├── Var('x') → tab:9, type:1
    │   ├── CaseBranch
    │   │   ├── IntLit(1) → type:1
    │   │   └── Assign → type:8
    │   │       ├── Var('y') → tab:10, type:2
    │   │       └── RealLit(1) → type:2
    │   └── CaseBranch
    │       ├── IntLit(2) → type:1
    │       └── Assign → type:8
    │           ├── Var('y') → tab:10, type:2
    │           └── RealLit(2) → type:2
    ├── Assign → type:8
    │   ├── Var('b') → tab:11, type:3
    │   └── BinOp('>') → type:3
    │       ├── Var('x') → tab:9, type:1
    │       └── IntLit(0) → type:1
```

```
    │       └── IntLit(0) → type:1
    ├── Assign → type:8
    │   ├── Var('b') → tab:11, type:3
    │   └── UnaryOp('not') → type:3
    │       └── Var('b') → tab:11, type:3
    ├── Assign → type:8
    │   ├── Var('b') → tab:11, type:3
    │   └── BinOp('and') → type:3
    │       ├── BinOp('==') → type:3
    │       │   ├── Var('x') → tab:9, type:1
    │       │   └── IntLit(1) → type:1
    │       └── BinOp('==') → type:3
    │           ├── Var('y') → tab:10, type:2
    │           └── RealLit(2) → type:2
    └── Assign → type:8
        ├── Var('b') → tab:11, type:3
        └── BinOp('or') → type:3
            ├── BinOp('==') → type:3
            │   ├── Var('x') → tab:9, type:1
            │   └── IntLit(1) → type:1
            └── BinOp('==') → type:3
                ├── Var('y') → tab:10, type:2
                └── RealLit(2) → type:2
```

		 === TAB (Symbol Table) === idx name obj typ ref nrm lev adr link 1 Real 2 2 0 1 0 0 -1 2 Integer 2 1 0 1 0 0 1 3 Char 2 4 0 1 0 0 2 4 Boolean 2 3 0 1 0 0 3 5 String 2 5 0 1 0 0 4 6 True 1 3 0 1 0 1 5 7 False 1 3 0 1 0 0 6 8 ControlFlow 5 8 0 1 0 0 -1 9 x 0 1 0 1 0 0 8 10 y 0 2 0 1 0 1 9 11 b 0 3 0 1 0 2 10 === BTAB (Block Table) === idx last lpar psize vsize 0 7 0 0 0 1 11 0 0 0 === ATAB (Array Table) === idx xtyp etyp eref low high elsz size
5	 program NestedScope; var x: integer; procedure outer; var y: integer; procedure inner; var z: integer; begin z := x + y; end; begin y := 10; inner; end; begin x := 5; outer; end.	 └── Program('NestedScope') → tab:8, type:8 ├── VarDecl('x') → tab:9, type:1 └── SimpleType('integer') ├── ProcDecl('outer') → tab:10, type:8, ref:2 └── Block ├── VarDecl('y') → tab:11, type:1, lev:1 └── SimpleType('integer') ├── ProcDecl('inner') → tab:12, type:8, lev:1, ref:3 └── Block ├── VarDecl('z') → tab:13, type:1, lev:2 └── SimpleType('integer') └── Compound └── Assign → type:8 ├── Var('z') → tab:13, type:1, lev:2 └── BinOp('+') → type:1, lev:2 ├── Var('x') → tab:9, type:1 └── Var('y') → tab:11, type:1, lev:1 └── Compound ├── Assign → type:8 ├── Var('y') → tab:11, type:1, lev:1 └── IntLit(10) → type:1, lev:1 └── ProcCall('inner') → tab:12, type:8 └── Compound ├── Assign → type:8 ├── Var('x') → tab:9, type:1 └── IntLit(5) → type:1 └── ProcCall('outer') → tab:10, type:8  === TAB (Symbol Table) === idx name obj typ ref nrm lev adr link 1 Real 2 2 0 1 0 0 -1 2 Integer 2 1 0 1 0 0 1 3 Char 2 4 0 1 0 0 2 4 Boolean 2 3 0 1 0 0 3 5 String 2 5 0 1 0 0 4 6 True 1 3 0 1 0 1 5 7 False 1 3 0 1 0 0 6 8 NestedScope 5 8 0 1 0 0 -1 9 x 0 1 0 1 0 0 8 10 outer 3 8 2 1 0 0 9 11 y 0 1 0 1 1 0 -1 12 inner 3 8 3 1 1 0 11 13 z 0 1 0 1 2 0 -1 === BTAB (Block Table) === idx last lpar psize vsize 0 7 0 0 0 1 10 0 0 0 2 12 -1 0 0 3 13 -1 0 0 === ATAB (Array Table) === idx xtyp etyp eref low high elsz size

6

```
program AssignCompat;
var
  i: integer;
  r: real;
begin
  i := 5;
  r := i;
  r := 3.14;
  i := 10 + 20;
  r := i + 3.14;
end.
```

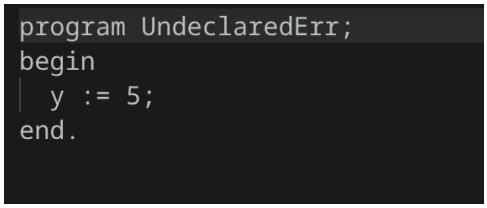
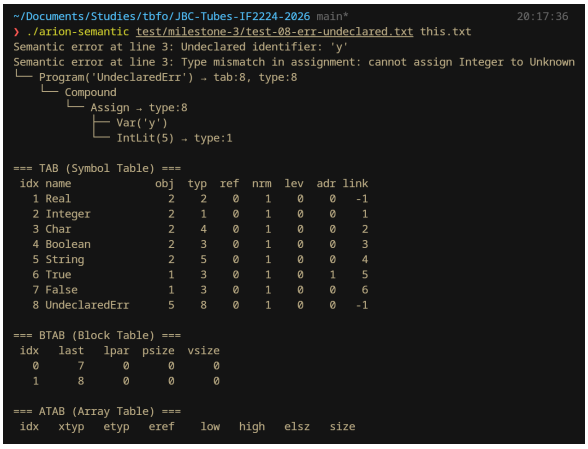
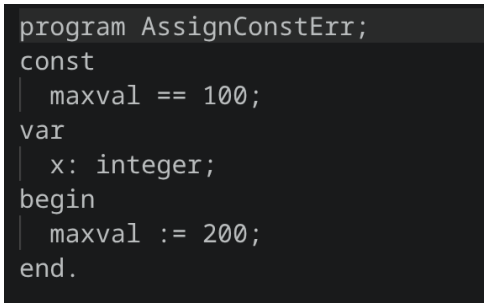
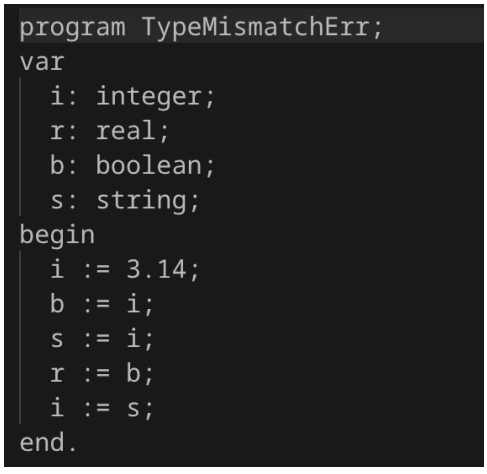
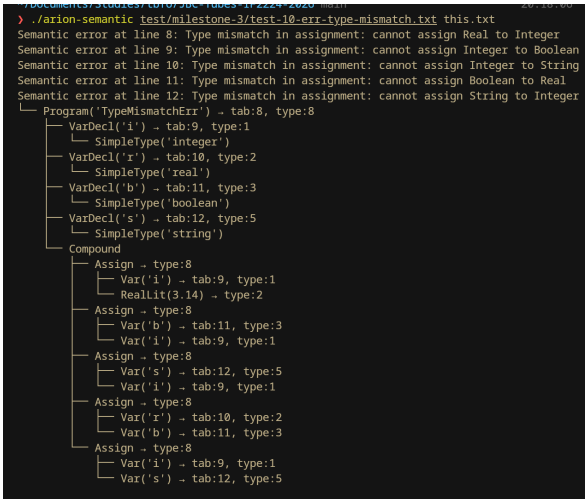
```
└── Program('AssignCompat') → tab:8, type:8
    ├── VarDecl('i') → tab:9, type:1
    │   └── SimpleType('integer')
    ├── VarDecl('r') → tab:10, type:2
    │   └── SimpleType('real')
    └── Compound
        ├── Assign → type:8
        │   ├── Var('i') → tab:9, type:1
        │   └── IntLit(5) → type:1
        ├── Assign → type:8
        │   ├── Var('r') → tab:10, type:2
        │   └── Var('i') → tab:9, type:1
        ├── Assign → type:8
        │   ├── Var('r') → tab:10, type:2
        │   └── RealLit(3.14) → type:2
        ├── Assign → type:8
        │   ├── Var('i') → tab:9, type:1
        │   └── BinOp('+') → type:1
        │       ├── IntLit(10) → type:1
        │       └── IntLit(20) → type:1
        └── Assign → type:8
            ├── Var('r') → tab:10, type:2
            └── BinOp('+') → type:2
                ├── Var('i') → tab:9, type:1
                └── RealLit(3.14) → type:2
```

```
=== TAB (Symbol Table) ===
 idx name             obj  typ  ref  nrm  lev  adr link
   1 Real               2    2    0    1    0    0   -1
   2 Integer            2    1    0    1    0    0    1
   3 Char               2    4    0    1    0    0    2
   4 Boolean            2    3    0    1    0    0    3
   5 String             2    5    0    1    0    0    4
   6 True               1    3    0    1    0    1    5
   7 False              1    3    0    1    0    0    6
   8 AssignCompat       5    8    0    1    0    0   -1
   9 i                  0    1    0    1    0    0    8
  10 r                  0    2    0    1    0    1    9

=== BTAB (Block Table) ===
 idx   last   lpar  psize  vsize
   0      7      0      0      0
   1     10      0      0      0

=== ATAB (Array Table) ===
 idx   xtyp   etyp   eref    low   high   elsz   size
```

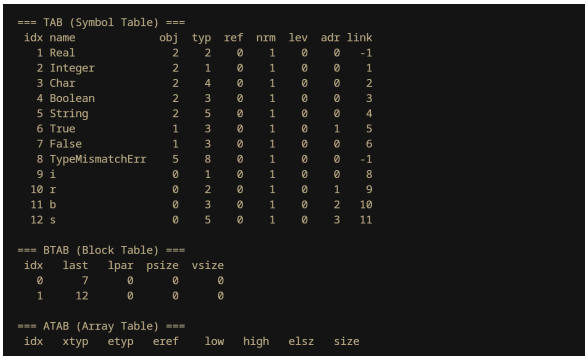
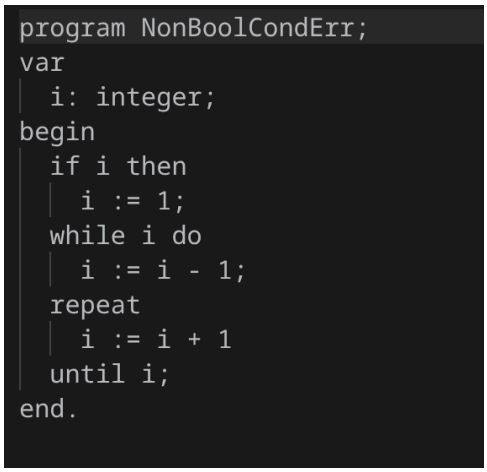
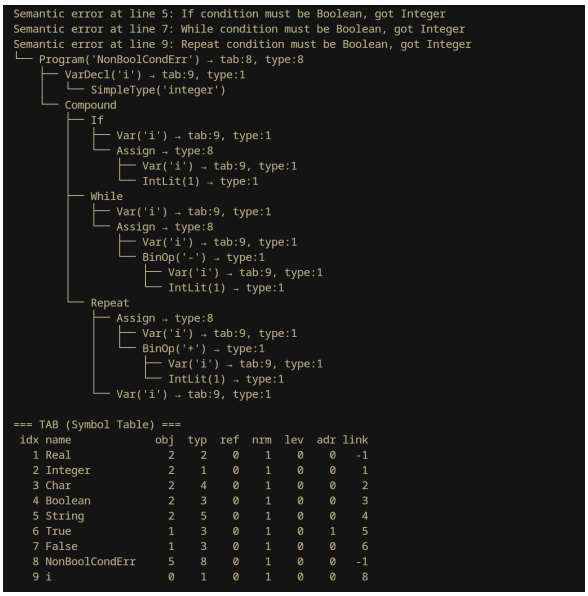
7

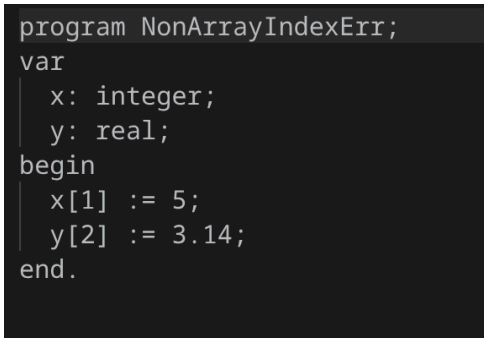
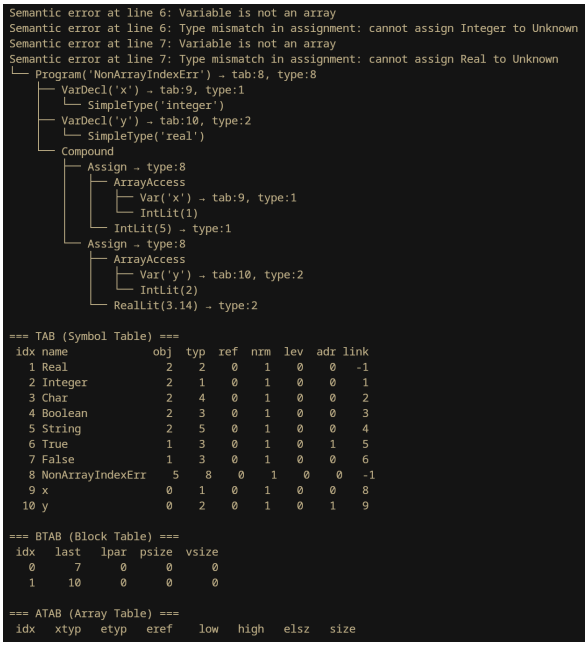
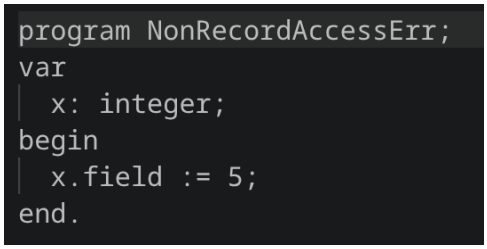
```
program RedeclareErr;
var
  x: integer;
  x: real;
begin
end.
```

```
❯ ./arion-semantic test/milestone-3/test-07-err-redeclare.txt this.txt
Semantic error at line 4: Identifier 'x' already declared in this scope
└── Program('RedeclareErr') → tab:8, type:8
    ├── VarDecl('x') → tab:9, type:1
    │   └── SimpleType('integer')
    ├── VarDecl('x')
    │   └── SimpleType('real')
    └── Compound

=== TAB (Symbol Table) ===
 idx name             obj  typ  ref  nrm  lev  adr link
   1 Real               2    2    0    1    0    0   -1
   2 Integer            2    1    0    1    0    0    1
   3 Char               2    4    0    1    0    0    2
   4 Boolean            2    3    0    1    0    0    3
   5 String             2    5    0    1    0    0    4
   6 True               1    3    0    1    0    1    5
   7 False              1    3    0    1    0    0    6
   8 RedeclareErr       5    8    0    1    0    0   -1
   9 x                  0    1    0    1    0    0    8

=== BTAB (Block Table) ===
 idx   last   lpar  psize  vsize
   0      7      0      0      0
   1      9      0      0      0

=== ATAB (Array Table) ===
 idx   xtyp   etyp   eref    low   high   elsz   size
```

8		
9		
10		

program UndeclaredErr;
begin
y := 5;
end.
~/Documents/Studies/tbfo/JBC-Tubes-IF2224-2026 main* 20:17:36
❯ ./arion-semantic test/milestone-3/test-08-err-undeclared.txt this.txt
Semantic error at line 3: Undeclared identifier: 'y'
Semantic error at line 3: Type mismatch in assignment: cannot assign Integer to Unknown
└── Program('UndeclaredErr') → tab:8, type:8
└── Compound
└── Assign → type:8
├── Var('y')
└── IntLit(5) → type:1
=== TAB (Symbol Table) ===
idx name obj typ ref nrm lev adr link
1 Real 2 2 0 1 0 0 -1
2 Integer 2 1 0 1 0 0 1
3 Char 2 4 0 1 0 0 2
4 Boolean 2 3 0 1 0 0 3
5 String 2 5 0 1 0 0 4
6 True 1 3 0 1 0 1 5
7 False 1 3 0 1 0 0 6
8 UndeclaredErr 5 8 0 1 0 0 -1
=== BTAB (Block Table) ===
idx last lpar psize vsize
0 7 0 0 0
1 8 0 0 0
=== ATAB (Array Table) ===
idx xtyp etyp eref low high elsz size
program AssignConstErr;
const
maxval == 100;
var
x: integer;
begin
maxval := 200;
end.
❯ ./arion-semantic test/milestone-3/test-09-err-assign-const.txt this.txt
Semantic error at line 7: Cannot assign to constant 'maxval'
└── Program('AssignConstErr') → tab:8, type:8
├── ConstDecl('maxval') → tab:9, type:1
│ └── IntLit(100) → type:1
├── VarDecl('x') → tab:10, type:1
│ └── SimpleType('integer')
└── Compound
└── Assign → type:8
├── Var('maxval') → tab:9, type:1
└── IntLit(200) → type:1
=== TAB (Symbol Table) ===
idx name obj typ ref nrm lev adr link
1 Real 2 2 0 1 0 0 -1
2 Integer 2 1 0 1 0 0 1
3 Char 2 4 0 1 0 0 2
4 Boolean 2 3 0 1 0 0 3
5 String 2 5 0 1 0 0 4
6 True 1 3 0 1 0 1 5
7 False 1 3 0 1 0 0 6
8 AssignConstErr 5 8 0 1 0 0 -1
9 maxval 1 1 0 1 0 100 8
10 x 0 1 0 1 0 0 9
=== BTAB (Block Table) ===
idx last lpar psize vsize
0 7 0 0 0
1 10 0 0 0
=== ATAB (Array Table) ===
idx xtyp etyp eref low high elsz size
program TypeMismatchErr;
var
i: integer;
r: real;
b: boolean;
s: string;
begin
i := 3.14;
b := i;
s := i;
r := b;
i := s;
end.
❯ ./arion-semantic test/milestone-3/test-10-err-type-mismatch.txt this.txt
Semantic error at line 8: Type mismatch in assignment: cannot assign Real to Integer
Semantic error at line 9: Type mismatch in assignment: cannot assign Integer to Boolean
Semantic error at line 10: Type mismatch in assignment: cannot assign Integer to String
Semantic error at line 11: Type mismatch in assignment: cannot assign Boolean to Real
Semantic error at line 12: Type mismatch in assignment: cannot assign String to Integer
└── Program('TypeMismatchErr') → tab:8, type:8
├── VarDecl('i') → tab:9, type:1
│ └── SimpleType('integer')
├── VarDecl('r') → tab:10, type:2
│ └── SimpleType('real')
├── VarDecl('b') → tab:11, type:3
│ └── SimpleType('boolean')
├── VarDecl('s') → tab:12, type:5
│ └── SimpleType('string')
└── Compound
├── Assign → type:8
│ ├── Var('i') → tab:9, type:1
│ └── RealLit(3.14) → type:2
├── Assign → type:8
│ ├── Var('b') → tab:11, type:3
│ └── Var('i') → tab:9, type:1
├── Assign → type:8
│ ├── Var('s') → tab:12, type:5
│ └── Var('i') → tab:9, type:1
├── Assign → type:8
│ ├── Var('r') → tab:10, type:2
│ └── Var('b') → tab:11, type:3
└── Assign → type:8
├── Var('i') → tab:9, type:1
└── Var('s') → tab:12, type:5

		 === TAB (Symbol Table) === idx name obj typ ref nrm lev adr link 1 Real 2 2 0 1 0 0 -1 2 Integer 2 1 0 1 0 0 1 3 Char 2 4 0 1 0 0 2 4 Boolean 2 3 0 1 0 0 3 5 String 2 5 0 1 0 0 4 6 True 1 3 0 1 0 1 5 7 False 1 3 0 1 0 0 6 8 TypeMismatchErr 5 8 0 1 0 0 -1 9 i 0 1 0 1 0 0 8 10 r 0 2 0 1 0 1 9 11 b 0 3 0 1 0 2 10 12 s 0 5 0 1 0 3 11 === BTAB (Block Table) === idx last lpar psize vsize 0 7 0 0 0 1 12 0 0 0 === ATAB (Array Table) === idx xtyp etyp eref low high elsz size
11	 program NonBoolCondErr; var i: integer; begin if i then i := 1; while i do i := i - 1; repeat i := i + 1 until i; end.	 Semantic error at line 5: If condition must be Boolean, got Integer Semantic error at line 7: While condition must be Boolean, got Integer Semantic error at line 9: Repeat condition must be Boolean, got Integer └─ Program('NonBoolCondErr') → tab:8, type:8 ├─ VarDecl('i') → tab:9, type:1 └─ SimpleType('integer') └─ Compound ├─ If ├─ Var('i') → tab:9, type:1 └─ Assign → type:8 ├─ Var('i') → tab:9, type:1 └─ IntLit(1) → type:1 ├─ While ├─ Var('i') → tab:9, type:1 └─ Assign → type:8 ├─ Var('i') → tab:9, type:1 └─ BinOp('-') → type:1 ├─ Var('i') → tab:9, type:1 └─ IntLit(1) → type:1 └─ Repeat ├─ Assign → type:8 ├─ Var('i') → tab:9, type:1 └─ BinOp('+') → type:1 ├─ Var('i') → tab:9, type:1 └─ IntLit(1) → type:1 └─ Var('i') → tab:9, type:1 === TAB (Symbol Table) === idx name obj typ ref nrm lev adr link 1 Real 2 2 0 1 0 0 -1 2 Integer 2 1 0 1 0 0 1 3 Char 2 4 0 1 0 0 2 4 Boolean 2 3 0 1 0 0 3 5 String 2 5 0 1 0 0 4 6 True 1 3 0 1 0 1 5 7 False 1 3 0 1 0 0 6 8 NonBoolCondErr 5 8 0 1 0 0 -1 9 i 0 1 0 1 0 0 8 === BTAB (Block Table) === idx last lpar psize vsize 0 7 0 0 0 1 9 0 0 0 === ATAB (Array Table) === idx xtyp etyp eref low high elsz size

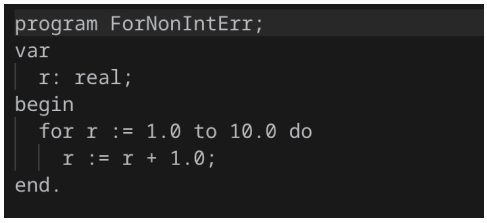
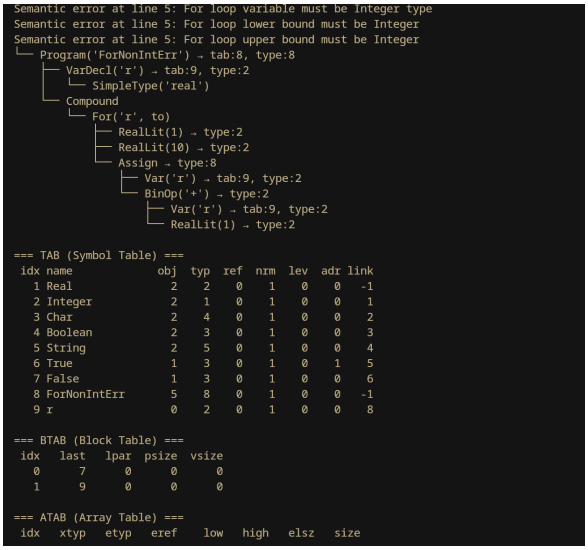
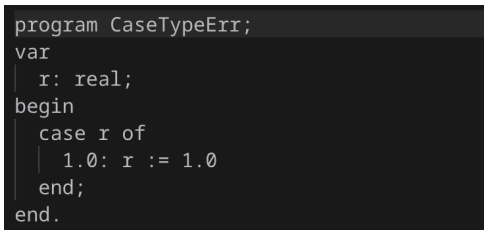
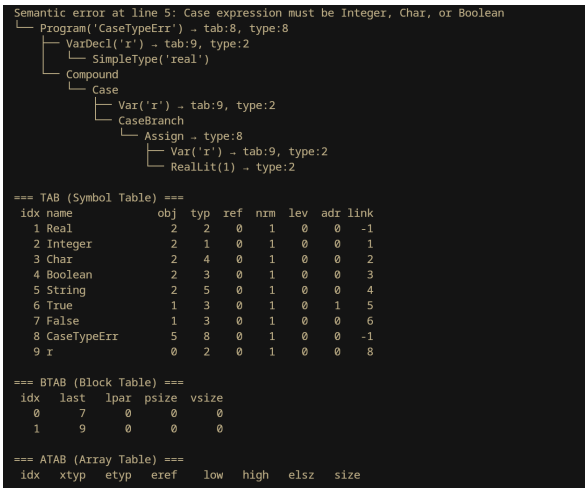
12	 program NonArrayIndexErr; var x: integer; y: real; begin x[1] := 5; y[2] := 3.14; end.	 Semantic error at line 6: Variable is not an array Semantic error at line 6: Type mismatch in assignment: cannot assign Integer to Unknown Semantic error at line 7: Variable is not an array Semantic error at line 7: Type mismatch in assignment: cannot assign Real to Unknown └─ Program('NonArrayIndexErr') → tab:8, type:8 ├─ VarDecl('x') → tab:9, type:1 └─ SimpleType('integer') ├─ VarDecl('y') → tab:10, type:2 └─ SimpleType('real') └─ Compound ├─ Assign → type:8 ├─ ArrayAccess ├─ Var('x') → tab:9, type:1 └─ IntLit(1) └─ IntLit(5) → type:1 └─ Assign → type:8 ├─ ArrayAccess ├─ Var('y') → tab:10, type:2 └─ IntLit(2) └─ RealLit(3.14) → type:2 === TAB (Symbol Table) === idx name obj typ ref nrm lev adr link 1 Real 2 2 0 1 0 0 -1 2 Integer 2 1 0 1 0 0 1 3 Char 2 4 0 1 0 0 2 4 Boolean 2 3 0 1 0 0 3 5 String 2 5 0 1 0 0 4 6 True 1 3 0 1 0 1 5 7 False 1 3 0 1 0 0 6 8 NonArrayIndexErr 5 8 0 1 0 0 -1 9 x 0 1 0 1 0 0 8 10 y 0 2 0 1 0 1 9 === BTAB (Block Table) === idx last lpar psize vsize 0 7 0 0 0 1 10 0 0 0 === ATAB (Array Table) === idx xtyp etyp eref low high elsz size
13	 program NonRecordAccessErr; var x: integer; begin x.field := 5; end.	 ./arion-semantic test/milestone-3/test-13-err-nonrecord-access.txt this.txt Semantic error at line 5: Variable is not a record Semantic error at line 5: Type mismatch in assignment: cannot assign Integer to Unknown └─ Program('NonRecordAccessErr') → tab:8, type:8 ├─ VarDecl('x') → tab:9, type:1 └─ SimpleType('integer') └─ Compound └─ Assign → type:8 ├─ RecordAccess('field') └─ Var('x') → tab:9, type:1 └─ IntLit(5) → type:1 === TAB (Symbol Table) === idx name obj typ ref nrm lev adr link 1 Real 2 2 0 1 0 0 -1 2 Integer 2 1 0 1 0 0 1 3 Char 2 4 0 1 0 0 2 4 Boolean 2 3 0 1 0 0 3 5 String 2 5 0 1 0 0 4 6 True 1 3 0 1 0 1 5 7 False 1 3 0 1 0 0 6 8 NonRecordAccessErr 5 8 0 1 0 0 -1 9 x 0 1 0 1 0 0 8 === BTAB (Block Table) === idx last lpar psize vsize 0 7 0 0 0 1 9 0 0 0 === ATAB (Array Table) === idx xtyp etyp eref low high elsz size ~/Documents/Studies/tbfo/JBC-Tubes-IF2224-2026 main* 20:19:31

14

```
program WrongArgsErr;
var
  r: integer;
procedure add(a: integer; b: integer);
begin
end;
function mul(a: integer; b: integer): integer;
begin
end;
begin
  add(1);
  add(1, 2, 3);
  r := mul(5);
  r := mul(1, 2, 3);
end.
```

```
❯ ./arion-semantic test/milestone-3/test-14-err-wrong-args.txt this.txt
Semantic error at line 11: Wrong number of arguments for 'add': expected 2, got 1
Semantic error at line 12: Wrong number of arguments for 'add': expected 2, got 3
Semantic error at line 13: Wrong number of arguments for 'mul': expected 2, got 1
Semantic error at line 14: Wrong number of arguments for 'mul': expected 2, got 3
└── Program('WrongArgsErr') → tab:8, type:8
    ├── VarDecl('r') → tab:9, type:1
    │   └── SimpleType('integer')
    ├── ProcDecl('add') → tab:10, type:8, ref:2
    │   ├── Param('a')
    │   │   └── SimpleType('integer')
    │   ├── Param('b')
    │   │   └── SimpleType('integer')
    │   └── Block
    │       └── Compound
    ├── FuncDecl('mul', returns: integer) → tab:13, type:1, ref:3
    │   ├── Param('a')
    │   │   └── SimpleType('integer')
    │   ├── Param('b')
    │   │   └── SimpleType('integer')
    │   └── Block
    │       └── Compound
    └── Compound
        ├── ProcCall('add') → tab:10, type:8
        │   └── IntLit(1) → type:1
        ├── ProcCall('add') → tab:10, type:8
        │   ├── IntLit(1) → type:1
        │   ├── IntLit(2) → type:1
        │   └── IntLit(3) → type:1
        ├── Assign → type:8
        │   ├── Var('r') → tab:9, type:1
        │   └── FuncCall('mul') → tab:13, type:1
        │       └── IntLit(5) → type:1
        └── Assign → type:8
            ├── Var('r') → tab:9, type:1
            └── FuncCall('mul') → tab:13, type:1
                ├── IntLit(1) → type:1
                ├── IntLit(2) → type:1
                └── IntLit(3) → type:1
```

```
=== TAB (Symbol Table) ===
 idx name            obj  typ  ref  nrm  lev  adr link
   1 Real              2    2    0    1    0    0   -1
   2 Integer           2    1    0    1    0    0    1
   3 Char              2    4    0    1    0    0    2
   4 Boolean           2    3    0    1    0    0    3
   5 String            2    5    0    1    0    0    4
   6 True              1    3    0    1    0    1    5
   7 False             1    3    0    1    0    0    6
   8 WrongArgsErr      5    8    0    1    0    0   -1
   9 r                 0    1    0    1    0    0    8
  10 add               3    8    2    1    0    0    9
  11 a                 0    1    0    1    1    0   -1
  12 b                 0    1    0    1    1    1   11
  13 mul               4    1    3    1    0    0   10
  14 a                 0    1    0    1    1    0   -1
  15 b                 0    1    0    1    1    1   14

=== BTAB (Block Table) ===
 idx   last   lpar  psize  vsize
   0      7      0      0      0
   1     13      0      0      0
   2     12     12      2      0
   3     15     15      2      0

=== ATAB (Array Table) ===
 idx   xtyp   etyp   eref    low   high   elsz   size
```

15	 program ForNonIntErr; var r: real; begin for r := 1.0 to 10.0 do r := r + 1.0; end.	 Semantic error at line 5: For loop variable must be Integer type Semantic error at line 5: For loop lower bound must be Integer Semantic error at line 5: For loop upper bound must be Integer └─ Program('ForNonIntErr') → tab:8, type:8 ├─ VarDecl('r') → tab:9, type:2 └─ SimpleType('real') └─ Compound └─ For('r', to) ├─ RealLit(1) → type:2 ├─ RealLit(10) → type:2 └─ Assign → type:8 ├─ Var('r') → tab:9, type:2 └─ BinOp('+') → type:2 ├─ Var('r') → tab:9, type:2 └─ RealLit(1) → type:2 === TAB (Symbol Table) === idx name obj typ ref nrm lev adr link 1 Real 2 2 0 1 0 0 -1 2 Integer 2 1 0 1 0 0 1 3 Char 2 4 0 1 0 0 2 4 Boolean 2 3 0 1 0 0 3 5 String 2 5 0 1 0 0 4 6 True 1 3 0 1 0 1 5 7 False 1 3 0 1 0 0 6 8 ForNonIntErr 5 8 0 1 0 0 -1 9 r 0 2 0 1 0 0 8 === BTAB (Block Table) === idx last lpar psize vsize 0 7 0 0 0 1 9 0 0 0 === ATAB (Array Table) === idx xtyp etyp eref low high elsz size
16	 program CaseTypeErr; var r: real; begin case r of 1.0: r := 1.0 end; end.	 Semantic error at line 5: Case expression must be Integer, Char, or Boolean └─ Program('CaseTypeErr') → tab:8, type:8 ├─ VarDecl('r') → tab:9, type:2 └─ SimpleType('real') └─ Compound └─ Case ├─ Var('r') → tab:9, type:2 └─ CaseBranch └─ Assign → type:8 ├─ Var('r') → tab:9, type:2 └─ RealLit(1) → type:2 === TAB (Symbol Table) === idx name obj typ ref nrm lev adr link 1 Real 2 2 0 1 0 0 -1 2 Integer 2 1 0 1 0 0 1 3 Char 2 4 0 1 0 0 2 4 Boolean 2 3 0 1 0 0 3 5 String 2 5 0 1 0 0 4 6 True 1 3 0 1 0 1 5 7 False 1 3 0 1 0 0 6 8 CaseTypeErr 5 8 0 1 0 0 -1 9 r 0 2 0 1 0 0 8 === BTAB (Block Table) === idx last lpar psize vsize 0 7 0 0 0 1 9 0 0 0 === ATAB (Array Table) === idx xtyp etyp eref low high elsz size

BAB 4
KESIMPULAN DAN SARAN

4.1.Kesimpulan

Semantic analyzer yang dikembangkan telah berhasil melengkapi tahapan kompilasi dengan melakukan validasi makna terhadap struktur sintaksis yang dihasilkan pada tahap sebelumnya. Modul mengadopsi pola design Visitor dengan skema L-Attributed Grammar untuk melakukan penelusuran Abstract Syntax Tree (AST) secara depth-first untuk memeriksa aturan tipe dan scope. Integrasi struktur symbol table yang terpartisi menjadi tab, atab, dan btab bersifat efektif dalam memetakan deklarasi identifier dan mengelola konteks eksekusi secara terperinci dan terorganisasi. Sistem mampu mengidentifikasi kesalahan logika semantik, seperti *type mismatch*, duplikasi deklarasi, atau akses variabel di luar scope, dan menghasilkan decorated AST yang memaparkan rincian atribut setiap node sebagai basis informasi yang akan digunakan pada tahapan selanjutnya.

4.2.Saran

Pengembangan komponen compiler seperti semantic analyzer membutuhkan pendefinisian masalah, spesifikasi bahasa, dan rancangan perilaku compiler yang jelas agar program dapat bekerja dengan andal dan tidak ambigu. Dengan penulisan spesifikasi yang lebih konsisten dan diranncang denagn matang, program dapat dibuat agar lebih sesuai denagn perilaku yang diharapkan. Ketika terdapat spesifikasi yang multitafsir atau membutuhkan konfirmasi, respons yang cepat tanggap dari panitia pemberi spesifikasi dapat meningkatkan efektivitas pengembangan program.

BAB 5

LAMPIRAN

5.1.Link Release Repository GitHub

https://github.com/samuelsondhrm/JBC-Tubes-IF2224-2026

5.2.Grammar yang Digunakan

Tabel 5.2. Definisi formal grammar program syntax analyzer bahasa pemrograman Arion

Komponen	**Anggota**
Variables	<program> <program-header> <declaration-part> <block> <const-declaration> <constant> <type-declaration> <type> <array-type> <range> <enumerated> <record-type> <field-list> <field-part> <var-declaration> <identifier-list> <subprogram-declaration> <procedure-declaration> <function-declaration> <formal-parameter-list> <parameter-group> <compound-statement> <statement-list> <statement> <variable> <component-variable> <index-list> <assignment-statement> <if-statement> <case-statement> <case-block> <while-statement> <repeat-statement> <for-statement> <procedure/function-call>

	<parameter-list> <expression> <simple-expression> <term> <factor> <relational-operator> <additive-operator> <multiplicative-operator>
Terminal	KEYWORD(program), KEYWORD(const), KEYWORD(type), KEYWORD(var), KEYWORD(array), KEYWORD(of), KEYWORD(record), KEYWORD(end), KEYWORD(procedure), KEYWORD(function), KEYWORD(begin), KEYWORD(if), KEYWORD(then), KEYWORD(else), KEYWORD(case), KEYWORD(while), KEYWORD(do), KEYWORD(repeat), KEYWORD(until), KEYWORD(for), KEYWORD(to), KEYWORD(downto), IDENTIFIER, INTEGER_LITERAL, REAL_LITERAL, CHAR_LITERAL, STRING_LITERAL, SEMICOLON(;), COLON(:), COMMA(,), PERIOD(.), LPARENTHESIS((), RPARENTHESIS()), LBRACKET([), RBRACKET(]), RELATIONAL_OPERATOR(==), RELATIONAL_OPERATOR(<>), RELATIONAL_OPERATOR(<), RELATIONAL_OPERATOR(<=), RELATIONAL_OPERATOR(>), RELATIONAL_OPERATOR(>=),

	ASSIGN_OPERATOR(:=), ARITHMETIC_OPERATOR(+), ARITHMETIC_OPERATOR(-), ARITHMETIC_OPERATOR(*), ARITHMETIC_OPERATOR(/), ARITHMETIC_OPERATOR(div), ARITHMETIC_OPERATOR(mod), LOGICAL_OPERATOR(and), LOGICAL_OPERATOR(or), LOGICAL_OPERATOR(not)
Productions	<program> ::= <program-header> <declaration-part> <compound-statement> PERIOD <program-header> ::= KEYWORD(program) IDENTIFIER SEMICOLON <declaration-part> ::= { <const-declaration> } { <type-declaration> } { <var-declaration> } { <subprogram-declaration> } <block> ::= <declaration-part> <compound-statement> <const-declaration> ::= KEYWORD(const) (IDENTIFIER RELATIONAL_OPERATOR(==) <constant> SEMICOLON)+ <constant> ::= CHAR_LITERAL | STRING_LITERAL | [ARITHMETIC_OPERATOR(+) | ARITHMETIC_OPERATOR(-)] (IDENTIFIER | INTEGER_LITERAL | REAL_LITERAL) <type-declaration> ::= KEYWORD(type) (IDENTIFIER RELATIONAL_OPERATOR(==) <type> SEMICOLON)+ <type> ::= <array-type> | <enumerated> | <record-type> | IDENTIFIER | <range> <array-type> ::= KEYWORD(array) LBRACKET (<range> | IDENTIFIER) RBRACKET KEYWORD(of) <type> <range> ::= <expression> PERIOD PERIOD <expression> <enumerated> ::= LPARENTHESIS IDENTIFIER { COMMA IDENTIFIER } RPARENTHESIS

<record-type> ::= KEYWORD(record) <field-list> KEYWORD(end)

<field-list> ::= <field-part> { SEMICOLON <field-part> } [SEMICOLON]

<field-part> ::= <identifier-list> COLON <type>

<var-declaration> ::= KEYWORD(var)
(<identifier-list> COLON <type> SEMICOLON)+

<identifier-list> ::= IDENTIFIER { COMMA IDENTIFIER }

<subprogram-declaration> ::= <procedure-declaration> | <function-declaration>

<procedure-declaration> ::= KEYWORD(procedure) IDENTIFIER [<formal-parameter-list>]
SEMICOLON <block> SEMICOLON

<function-declaration> ::= KEYWORD(function) IDENTIFIER [<formal-parameter-list>]
COLON IDENTIFIER SEMICOLON <block> SEMICOLON

<formal-parameter-list> ::= LPARENTHESIS <parameter-group>
{ SEMICOLON <parameter-group> } RPARENTHESIS

<parameter-group> ::= <identifier-list> COLON (<array-type> | IDENTIFIER)

<compound-statement> ::= KEYWORD(begin) <statement-list> KEYWORD(end)

<statement-list> ::= <statement> { SEMICOLON <statement> }

<statement> ::= <assignment-statement>
| <if-statement>
| <case-statement>
| <while-statement>
| <repeat-statement>
| <for-statement>
| <procedure/function-call>
| <compound-statement>
| ε

<variable> ::= IDENTIFIER { <component-variable> }

<component-variable> ::= LBRACKET <index-list> RBRACKET
| PERIOD IDENTIFIER

<index-list> ::= (INTEGER_LITERAL | CHAR_LITERAL | IDENTIFIER)
{ COMMA (INTEGER_LITERAL | CHAR_LITERAL | IDENTIFIER) }

<assignment-statement> ::= <variable> ASSIGN_OPERATOR(:=) <expression>

<if-statement> ::= KEYWORD(if) <expression> KEYWORD(then) <statement>
[KEYWORD(else) <statement>]

<case-statement> ::= KEYWORD(case) <expression> KEYWORD(of)
<case-block> KEYWORD(end)

<case-block> ::= <constant> { COMMA <constant> } COLON <statement>
{ SEMICOLON [<case-block>] }

<while-statement> ::= KEYWORD(while) <expression> KEYWORD(do) <statement>

<repeat-statement> ::= KEYWORD(repeat) <statement-list> KEYWORD(until) <expression>

<for-statement> ::= KEYWORD(for) IDENTIFIER ASSIGN_OPERATOR(:=) <expression>
(KEYWORD(to) | KEYWORD(downto)) <expression>
KEYWORD(do) <statement>

<procedure/function-call> ::= IDENTIFIER
[LPARENTHESIS [<parameter-list>] RPARENTHESIS]

<parameter-list> ::= <expression> { COMMA <expression> }

<expression> ::= <simple-expression>
[<relational-operator> <simple-expression>]

<simple-expression> ::= [ARITHMETIC_OPERATOR(+) | ARITHMETIC_OPERATOR(-)]
<term> { <additive-operator> <term> }

<term> ::= <factor> { <multiplicative-operator> <factor> }

	<factor> ::= <procedure/function-call> | <variable> | INTEGER_LITERAL | REAL_LITERAL | CHAR_LITERAL | STRING_LITERAL | LPARENTHESIS <expression> RPARENTHESIS | LOGICAL_OPERATOR(not) <factor> <relational-operator> ::= RELATIONAL_OPERATOR(==) | RELATIONAL_OPERATOR(<>) | RELATIONAL_OPERATOR(<) | RELATIONAL_OPERATOR(<=) | RELATIONAL_OPERATOR(>) | RELATIONAL_OPERATOR(>=) <additive-operator> ::= ARITHMETIC_OPERATOR(+) | ARITHMETIC_OPERATOR(-) | LOGICAL_OPERATOR(or) <multiplicative-operator> ::= ARITHMETIC_OPERATOR(*) | ARITHMETIC_OPERATOR(/) | ARITHMETIC_OPERATOR(div) | ARITHMETIC_OPERATOR(mod) | LOGICAL_OPERATOR(and)
Start	<program>

5.3.Pembagian Tugas

Tabel 5.3. Pembagian tugas

NIM	**Pembagian tugas**	**Persentase**
13524001	1. Implementasi ASTBuilder, ASTNode, DecoratedASTPrinter 2. Laporan	20%
13524027	1. Implementasi SymbolTable 2. Laporan	20%
13524029	1. Implementasi visit_declarations 2. Laporan	20%

13524089	1. Implementasi visit_expression, visit_statement 2. Laporan	20%
13524093	1. Implementasi TypeChecker 2. Laporan	20%

5.4.Referensi

Aho, Alfred V, et al. *Compilers : Principles, Techniques, and Tools*. Noida, India, Dorling Kindersley (India) Pvt. Ltd., Licensees Of Pearson Education In South Asia, 2014.

Spivak, Ruslan. "Let's Build A Simple Interpreter. Part 13." *Ruslan's Blog*, 2015, https://ruslanspivak.com/lsbasi-part13/